

1.1 Understanding AI, machine learning, and deep learning

What is AI

Artificial intelligence (AI) is a branch of computer science that aims to create machines capable of mimicking human intelligence. Unlike traditional systems that follow explicit instructions, AI systems are designed to process information and make decisions or predictions based on the data they're given. The overarching goal of AI is to develop algorithms and models that allow machines to perform tasks—ranging from recognizing patterns to decision-making—that would usually require human cognition. AI's scope spans various technologies, including robotics, natural language processing (NLP), and expert systems. Its applications are evident in daily life, with systems such as virtual assistants, facial recognition software, and autonomous vehicles. AI's impact is transformative, redefining how industries operate and how we interact with technology.

Historical development The journey of AI began in the 1940s and 1950s with the development of the first electronic computers. The 1980s saw the rise of machine learning (ML), where algorithms learn directly from data rather than relying on explicit programming. Neural networks, a subset of ML, faced challenges until the 2000s when computational power and data availability grew. This resurgence, now termed deep learning, uses multilayered neural networks to process vast datasets. The game-changing breakthroughs, such as Deep Blue's chess victory in 1997 and AlphaGo's win in 2016, marked significant milestones. Today, AI encompasses a blend of these techniques, continuously evolving with advancements in computation, data, and algorithms.

Applications of AI AI has woven its way into a multitude of sectors, revolutionizing processes and augmenting human capabilities. In health care, AI algorithms are being used to diagnose diseases, sometimes with accuracy surpassing human doctors. In finance, it powers fraud detection systems, optimizing security. The automotive industry is witnessing a transformation with AI-driven autonomous vehicles. In entertainment, recommendation systems such as those in Netflix or Spotify customize user experiences. E-commerce platforms use AI for predicting consumer behavior, enhancing sales strategies. Virtual assistants such as Siri and Alexa employ AI to comprehend and respond to user commands. In manufacturing, AI-driven robots optimize assembly lines, increasing efficiency. Additionally, in the realm of research, AI aids in complex simulations and data analysis. From smart homes to predictive text on smartphones, the applications of AI are vast, continuously expanding, and making an indelible mark on how society functions and evolves.

AI today The current landscape of AI is characterized by rapid advancements and widespread adoption across various sectors. Breakthroughs in machine learning, especially deep learning, have propelled AI capabilities, making tasks such as image and speech recognition more accurate than ever before. AI models, such as GPT-3 and BERT, have revolutionized natural language processing, enabling seamless human-computer interactions. The growth of big data and enhanced computational power, through GPUs,

has further accelerated AI research and applications. Today's businesses leverage AI for predictive analytics, customer insights, and automation. Ethical concerns, such as biases in AI models and privacy issues, have prompted discussions and regulations. Innovations in AI have also sparked debates on the future of employment, as automation replaces certain job functions. However, alongside challenges, AI offers immense potential to drive efficiency, innovation, and growth in the 21st century.

The future of AI The future of AI holds immense potential and is poised to be transformational across various domains. As AI algorithms become more sophisticated, we'll see further personalization in services, from tailored education platforms to individualized health monitoring. The continued convergence of AI with fields such as quantum computing could redefine computational limits, allowing for the solving of currently insurmountable problems. Ethical considerations will gain prominence, with emphasis on transparency, fairness, and avoiding biases in AI systems. There will also be a focus on achieving general AI, a system with cognitive abilities akin to human intelligence. As AI integrates more deeply with our daily lives, new job roles and industries will emerge, while others adapt or phase out. Lastly, international collaborations and regulations will play a crucial role in ensuring AI's safe and equitable development and deployment.

What is machine learning

Machine learning (ML) is a subset of artificial intelligence that focuses on the development of algorithms that allow computers to learn from and make decisions based on data. Rather than being explicitly programmed for a specific task, ML models use statistical techniques to understand patterns in data. By processing large amounts of data, these models can make predictions or decisions without human intervention. For example, a machine learning model can be trained to recognize images of cats by being shown many images of cats and non-cats. Over time, it fine-tunes its understanding and improves its accuracy. The essence of ML lies in its iterative nature; as more data becomes available, the model adjusts and evolves. This ability to learn from data makes machine learning integral in today's AI-driven world, fueling advancements in fields ranging from health care to finance.

What is deep learning

Deep learning is a specialized subset of machine learning inspired by the structure and function of the human brain, specifically neural networks. It employs artificial neural networks, especially deep neural networks with multiple layers, to analyze various factors of data. Deep learning models are particularly powerful for tasks such as image and speech recognition. For instance, when processing an image, the model might first identify edges, then shapes, and eventually complex features such as faces or objects. The "deep" in deep learning refers to the number of layers in the neural network. Traditional neural networks might contain two or three layers, while deep networks can have hundreds. These intricate architectures allow deep learning models to automatically extract features and learn intricate patterns from vast amounts of data, often outperforming other machine learning models in accuracy and efficiency, especially when dealing with large-scale data.

What is generative AI

Generative AI refers to a subset of artificial intelligence models that are designed to generate new data samples that are similar in nature to a given set of input data. In essence, these models "learn" the underlying patterns, structures, and features of input data and then use this knowledge to create entirely new data samples. The resulting outputs, whether they are images, texts, or sounds, are often indistinguishable from real-world data. A quintessential example is the generative adversarial network (GAN), where two neural networks—a generator and a discriminator—are pitted against each other. The generator strives to produce data, while the discriminator evaluates its authenticity. Through iterative training, the generator improves its outputs. Beyond GANs, other generative models such as variational autoencoders (VAEs) also find extensive applications in tasks such as image synthesis and style transfer. The appeal of generative AI lies in its potential to craft novel yet coherent creations by understanding and mimicking complex data distributions.

Early beginnings of generative AI The genesis of generative AI dates back to the mid-20th century, rooted in foundational statistical modeling and pattern recognition techniques. Early forms of generative models included Gaussian mixture models (GMMs) and hidden Markov models (HMMs), which were pivotal in speech recognition and computational biology. While these models demonstrated the concept of capturing data distributions, their real-world applications were somewhat limited due to computational constraints and the lack of vast datasets. However, the introduction of neural networks in the 1980s paved the way for more sophisticated generative models. The Boltzmann machine, an early form of a neural network with a generative structure, was one such breakthrough. By the 2000s, with the rise of computational power and the availability of large datasets, models such as restricted Boltzmann machines (RBMs) became feasible. These foundational steps were the precursors to the contemporary generative models, such as GANs and VAEs, which now drive much of today's AI-generated content.

The current evolution of generative AI Generative AI has experienced remarkable evolution in recent years, driven largely by advancements in neural network architectures and computational power. One of the pivotal moments was the introduction of GANs by Ian Goodfellow in 2014. As previously explained, GANs consist of two neural networks, the generator and discriminator, which work in tandem to produce highly realistic outputs. Variational autoencoders (VAEs) have also become a popular generative model, known for their probabilistic approach to generating new samples. These tools have facilitated groundbreaking applications such as creating realistic images, designing drug molecules, and even generating art and music. The surge in deepfake technology, which convincingly replaces faces in videos, underscores the power of these generative models. Additionally, transformer-based models, such as OpenAI's GPT series, have demonstrated the capability to generate humanlike text. The rapid progress in generative AI underscores its transformative potential and continuously blurs the line between human-generated and machine-generated content.

What are discriminative models Discriminative models, in the realm of machine learning, are primarily concerned with distinguishing between different classes or categories based on input data. Rather than capturing the data distribution like generative models, they focus on modeling the boundary separating different classes. For instance, in a binary classification problem, a discriminative model would aim to discern the boundary that separates two categories, enabling predictions about which class a new input belongs to. Common examples of discriminative algorithms include logistic regression, support vector machines, and most deep neural networks designed for classification tasks. They are often chosen for tasks where pinpointing the exact decision boundary is more crucial than understanding the underlying

data distribution. Discriminative models, given their direct approach, tend to be more accurate than generative models for classification tasks, but they don't offer insights into the characteristics or patterns that define each class.

Applications of generative AI Generative AI has revolutionized numerous fields with its ability to generate new, previously unseen content. In art and entertainment, GANs have been utilized to create realistic artwork, music, and even video game levels. In the fashion industry, generative models suggest novel clothing designs or adapt existing styles to personalized preferences. The health care sector benefits from synthesizing medical images for research, enhancing the training data pool without compromising patient privacy. In the realm of natural language processing, generative models, such as GPT variants, produce humanlike text, enabling more sophisticated chatbots and content creation tools. Additionally, in the realm of chemistry and drug discovery, generative models propose molecular structures for new potential drugs. Generative AI also aids in data augmentation, where limited datasets are expanded by creating variations, thus improving model training. These applications underscore generative AI's transformative potential across diverse sectors.

Limitations of generative AI Generative AI, despite its groundbreaking capabilities, possesses inherent limitations. Firstly, training generative models, especially advanced architectures such as GANs, demands considerable computational resources and time. This is not always feasible for individual developers or small entities. Secondly, these models can sometimes produce unrealistic or nonsensical outputs, especially when they encounter data significantly different from their training set. Another concern is the ethical implications of generative AI: the creation of deepfakes in videos or misleading information can have severe societal ramifications. Intellectual property rights can also be jeopardized when generative models produce content indistinguishable from human-made creations. Moreover, ensuring fairness and avoiding biases in outputs is challenging, as these models can inadvertently learn and perpetuate existing biases from their training data. Lastly, interpretability remains a challenge; understanding how these models arrive at particular outputs is not always straightforward, which can hinder trust and widespread adoption.

The future of generative AI Generative AI stands at the precipice of a transformative future, redefining various industries and societal interactions. As computational power advances and algorithms refine, we anticipate more robust and efficient generative models. These models will likely produce outputs of higher fidelity, increasing their realism and utility. Integration with augmented reality (AR) and virtual reality (VR) environments could revolutionize the entertainment, gaming, and education sectors. Custom content creation, tailored to individual preferences, will become commonplace, personalizing user experiences like never before. Ethical considerations will take center stage, prompting the development of regulatory frameworks and tools to detect AI-generated content, combating misinformation and unauthorized reproductions. Additionally, advancements in semi-supervised and unsupervised learning will make generative AI more accessible, reducing the need for vast labeled datasets. Collaborative efforts between AI researchers and domain experts will further broaden the horizons, unlocking multifaceted applications that are currently unforeseen.

What is a language model?

Language models have undergone significant advancements over the past few years. At their core, these models are designed to understand and generate human language. Through different architectural approaches and training methods, researchers have developed several types of language models, each catering to specific needs and applications.

N-gram language models This is one of the earliest types of language models. An n-gram model predicts the next word in a sequence based on the $(n - 1)$ preceding words. For instance, a bigram (2-gram) model would consider two words at a time.

- **Usage:** N-gram models have been historically used in spell-check systems and basic text predictions.
- **Limitation:** These models struggle with long-term dependencies because they only consider the n previous words. Additionally, they do not scale well with increasing vocabulary sizes.

Recurrent Neural Networks (RNNs) RNNs process sequences of data by maintaining a *memory* from previous steps. This allows them to capture information from earlier in the sequence and use it to influence later predictions.

- **Usage:** RNNs have been employed in tasks such as machine translation, and sentiment analysis.
- **Limitation:** They can be computationally intensive and face challenges with very long sequences, often forgetting information from the earliest parts of the input.

Long Short-Term Memory (LSTM) networks LSTM is a special kind of RNN that includes a mechanism to remember and forget information selectively. This helps in tackling the long-term dependency problem seen in basic RNNs.

- **Usage:** LSTMs are widely used in time series forecasting, machine translation, and speech recognition.
- **Limitation:** While LSTMs mitigate some of the challenges of RNNs, they can still be computationally heavy, especially with very large datasets.

Transformer models Introduced in the paper "Attention Is All You Need," transformer models utilize self-attention mechanisms to weigh input data differently, enabling the model to focus on more relevant parts of the input for different tasks.

- **Usage:** Transformers have become the go-to architecture for many NLP tasks, including text generation, machine translation, and question answering.
- **Limitation:** The computational needs for transformer models are intense, necessitating powerful hardware setups, especially for large-scale models.

BERT (Bidirectional Encoder Representations from Transformers) BERT is a pretrained transformer model that considers the context from both the left and the right side of a word in all layers, making it deeply bidirectional.

- **Usage:** BERT and its variants have set state-of-the-art performance records on several NLP tasks such as sentiment analysis and named entity recognition.

- **Limitation:** Fine-tuning BERT for specific tasks can be computationally expensive. Additionally, its deep bidirectionality can make it less interpretable.

GPT (Generative Pretrained Transformer) Unlike BERT, which is trained to predict masked words in a sequence, GPT is trained to predict the next word in a sequence, making it a generative model.

- **Usage:** GPT models, especially GPT-3 by OpenAI, have demonstrated humanlike text generation capabilities, answering questions, writing essays, and even crafting poetry.
- **Limitation:** GPT models can sometimes generate plausible-sounding but incorrect or nonsensical outputs. They also require vast amounts of data for training.

Summary Language models have transitioned from simple statistical methods to complex neural network architectures. With each evolution, they've become more adept at understanding the intricacies of human language. However, each model type has its strengths and challenges, and the choice often depends on the specific application and available computational resources. As AI research advances, we can anticipate even more sophisticated models that seamlessly integrate with human linguistic interactions.

Applications in data management Data management, the practice of collecting, keeping, and using data securely, efficiently, and cost-effectively, is essential to businesses and organizations of all sizes. With the recent rise of sophisticated language models, there's been a transformative shift in how data management processes are executed. Here's a look at how language models are revolutionizing data management:

Data entry and cleaning Manual data entry and data cleaning are two of the most time-consuming tasks in data management. Language models can automate these processes by extracting information from unstructured sources such as emails, documents, and websites, converting them into structured formats. Additionally, they can identify and rectify inconsistencies, duplicates, and errors in datasets, ensuring data quality.

Semantic search Traditional search mechanisms rely on keyword matching, often returning irrelevant results. With language models, semantic search becomes possible, wherein the context and meaning of the query are understood. This ensures that database searches are not just keyword-based but contextually relevant, fetching more accurate and meaningful results.

Data classification and categorization Language models can automatically categorize and label vast amounts of data. For instance, customer feedback can be automatically sorted into categories such as positive, negative, or neutral. Similarly, documents can be classified based on their content, facilitating faster retrieval and better organization.

Natural language queries For those unfamiliar with SQL or other database querying languages, extracting specific data can be challenging. Language models allow users to fetch data using natural language queries. For instance, a user could ask, "Show me sales data for the last quarter," and the language model would translate that into an appropriate database query.

Content generation and summarization Language models can generate humanlike text based on data insights. For businesses, this could mean automatic report generation, where insights drawn from data

analytics are converted into understandable narratives. Additionally, models can summarize vast amounts of data, providing executives with concise briefs instead of lengthy reports.

Data privacy and redaction With rising concerns about data privacy, there's an increasing need to redact personal information from databases, especially when sharing datasets. Language models can automatically identify and mask sensitive information, ensuring data privacy compliance.

Chatbots and customer support Data management isn't just about handling internal data but also managing customer interactions. Language models power intelligent chatbots that can fetch information from databases in real time to answer customer queries, reducing the load on human agents and ensuring efficient data-driven customer service.

Predictive text and autocompletion For data managers and analysts, predictive text powered by language models can expedite data entry tasks. By predicting what the user intends to type next, these models can accelerate the data entry process, reducing manual effort and errors.

Multilingual data management In a globalized world, businesses often deal with data in multiple languages. Language models can automatically translate and transcribe data, ensuring seamless data management across linguistic barriers.

Insights and recommendations Language models, when combined with other AI techniques, can provide actionable insights by analyzing patterns and trends in data. For e-commerce businesses, this could mean product recommendations based on customer behavior and preferences.

In conclusion, language models are rapidly becoming a cornerstone of modern data management. By automating tasks, ensuring data quality, and facilitating human-AI collaboration, these models are streamlining data processes and enabling businesses to derive more value from their data. As they continue to evolve, the synergy between language models and data management promises even more innovative solutions and efficiencies.

Applications of AI in business

Health care AI emerges as a transformative force in health care, offering unprecedented opportunities for both care delivery and business processes. Several ways AI has been instrumental in health care include:

- **Disease identification and diagnosis:** Advanced AI algorithms analyze medical imaging such as X-rays, MRIs, and CT scans, aiding in the early detection and diagnosis of diseases such as cancer, allowing for timely interventions.
- **Treatment personalization:** AI analyzes patient data to recommend personalized treatment plans, taking into account the patient's genetic makeup, lifestyle, and other factors.
- **Drug discovery and development:** AI accelerates the drug development process by predicting how different compounds can treat diseases, significantly reducing the time and cost associated with traditional research.
- **Operational efficiency:** AI-powered systems streamline administrative tasks such as appointment scheduling, billing, and patient record maintenance, leading to enhanced operational efficiency.
- **Remote monitoring:** Wearable devices equipped with AI monitor vital statistics, alerting health care providers to potential health issues, enabling early intervention and reducing hospital readmissions.

For businesses within the health care sector, embracing AI equates to improved patient outcomes, reduced costs, and optimized operations. As AI continues to evolve, its potential to reshape health care delivery and its associated business models becomes increasingly evident.

Manufacturing AI stands at the forefront of the Fourth Industrial Revolution, reshaping the manufacturing landscape. The integration of AI in manufacturing yields several transformative benefits:

- **Predictive maintenance:** AI systems analyze machine data to predict when equipment is likely to fail, enabling timely maintenance. This reduces downtime, extending machinery life and decreasing operational costs.
- **Quality assurance:** Advanced vision systems powered by AI ensure product quality by identifying defects in real time on the production line, guaranteeing consistent product quality and reducing wastage.
- **Supply chain optimization:** AI algorithms process vast amounts of data to optimize inventory levels, predict demand, and enhance supply chain agility.
- **Smart robotics:** Robots, augmented with AI, can perform complex tasks, adapt to changes, and work collaboratively with humans, boosting production efficiency.
- **Energy consumption reduction:** AI-driven systems monitor and analyze energy usage patterns, optimizing consumption and leading to significant cost savings.

For businesses in the manufacturing domain, AI represents an avenue for innovation, operational excellence, and cost-efficiency. Its continued integration is set to further elevate manufacturing capabilities, driving industry growth.

Disaster management In the face of increasing global calamities, businesses are leveraging AI to fortify disaster management efforts, ensuring continuity, and safeguarding assets and human resources:

- **Early warning systems:** AI models process vast amounts of data from satellites, ocean buoys, and sensors to predict natural disasters such as hurricanes, earthquakes, or floods, allowing businesses to implement precautionary measures in a timely manner.
- **Resource allocation:** After a disaster, AI algorithms analyze the impact and distribute resources efficiently, ensuring urgent supplies reach the hardest-hit areas promptly.
- **Damage assessment:** AI-driven drones and satellite imagery help in assessing the extent of damage, assisting businesses in understanding the immediate implications on infrastructure, operations, and supply chains.
- **Rescue operations:** AI-enhanced robots are deployed in situations too hazardous for humans, ensuring swift rescue missions, especially in collapsed buildings or flood situations.
- **Business continuity planning:** AI assists businesses in creating robust continuity plans by simulating disaster scenarios, ensuring minimal disruptions during real-world events.

For businesses, AI's application in disaster management isn't merely a technological advancement; it's a crucial strategy to ensure resilience, safety, and sustainability in a volatile world.

Climate change Climate change presents a complex challenge, and businesses are turning to AI to both mitigate its effects and adapt to its evolving realities:

- **Predictive analysis:** Businesses are using AI to forecast environmental shifts and the implications they hold for industries. This helps firms in sectors such as agriculture, real estate, and insurance anticipate, prepare for, and navigate changes.
- **Carbon footprint reduction:** AI optimizes energy use in manufacturing processes, warehouses, and offices. By monitoring and adjusting energy consumption patterns, companies can reduce emissions and operational costs.
- **Supply chain resilience:** AI algorithms predict climate-induced disruptions and suggest alternatives, ensuring businesses maintain seamless operations even under unpredictable weather patterns.
- **Sustainable solutions development:** AI is aiding research in sustainable materials and renewable energy. Companies in the energy sector use it to optimize the output of solar panels and wind turbines.
- **Stakeholder engagement:** Businesses employ AI to analyze consumer sentiment, enabling them to align products and marketing strategies with growing demand for sustainability.

In the fight against climate change, AI empowers businesses to be proactive, making them part of the solution while ensuring long-term sustainability and resilience.

Economy AI is shaping the economic landscape, redefining the way businesses operate and driving economic growth:

- **Efficiency and automation:** Businesses are adopting AI-driven automation to streamline operations, reduce overhead costs, and enhance productivity. This leads to optimized business processes and increased competitiveness in the global market.
- **Financial analysis:** AI algorithms provide deeper insights into market trends, predicting stock market movements, and assisting businesses in making informed investment decisions. Furthermore, fintech companies leverage AI for fraud detection and credit risk assessment.
- **Supply chain optimization:** AI assists businesses in predicting demand, ensuring optimal stock levels, and minimizing wastage. This results in a more agile and responsive supply chain, adapting to market shifts.
- **Consumer personalization:** AI-driven analytics enable businesses to understand consumer preferences in real time, allowing for personalized product recommendations, which boost sales and enhance customer loyalty.
- **Job creation and evolution:** While there's concern over AI displacing jobs, it's also creating new roles and reshaping existing ones. Businesses are benefiting from a skilled workforce trained to harness the capabilities of AI.

In summary, AI acts as a catalyst in the economic sphere, promoting growth, enhancing efficiency, and redefining business operations.

1.2 Introduction to artificial neural networks

Learn to:

- Identify parts of a neuron.
- Explain how neurons communicate.
- Explain how artificial neurons connect inputs to outputs.
- Identify layers of an artificial neural network.

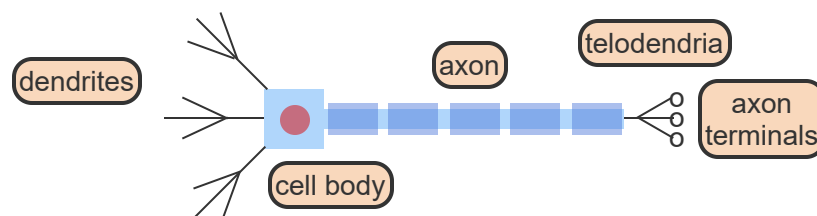
What is a neuron?

A **neuron** is a nerve cell that carries messages throughout the body. The main parts of a neuron are the cell body, axon, and dendrite.

- The **cell body** contains the nucleus, which has genetic information, maintains cellular function, and provides energy to the neuron.
- The **axon** is the long, thin part of the neuron in which electrical signals travel.
- The **dendrite** receives chemical information and determines whether an electrical signal is produced.

PARTICIPATION ACTIVITY

1.2.1: Parts of a neuron.



Animation content:

An image of a neuron. The dendrites are branches attached to the cell body that receive signals from another neuron. The cell body is connected to a long, thin fiber that carries signals away from the cell body with branches at the end called telodendria. Each telodendron has an axon terminal that releases neurotransmitters to another neuron.

Animation captions:

1. Dendrites are tree-like structures that receive chemical signals from other neurons. Dendrites convert these signals to electrical signals transmitted toward the cell body.
2. The cell body maintains various functions of the cell. Although the cell body keeps the neuron functioning, the cell body does not play an active role in sending signals.
3. The axon is a long fiber with branches at the end called telodendria. The axon carries electrical signals away from the cell body.
4. The axon terminals are located at the end of each telodendron. These terminals transmit chemical signals to other neurons.

PARTICIPATION ACTIVITY

1.2.2: Parts of a neuron.



1) The part of a neuron that receives information from other neurons is the ____.



- ☐ cell body
- ☐ axon
- ☐ dendrite

2) Chemical signals are transmitted ____.



- ☐ between neurons
- ☐ within a neuron

3) A neuron is the basic unit of the ____ system.



- ☐ nervous
- ☐ circulatory
- ☐ skeletal

How do neurons communicate?

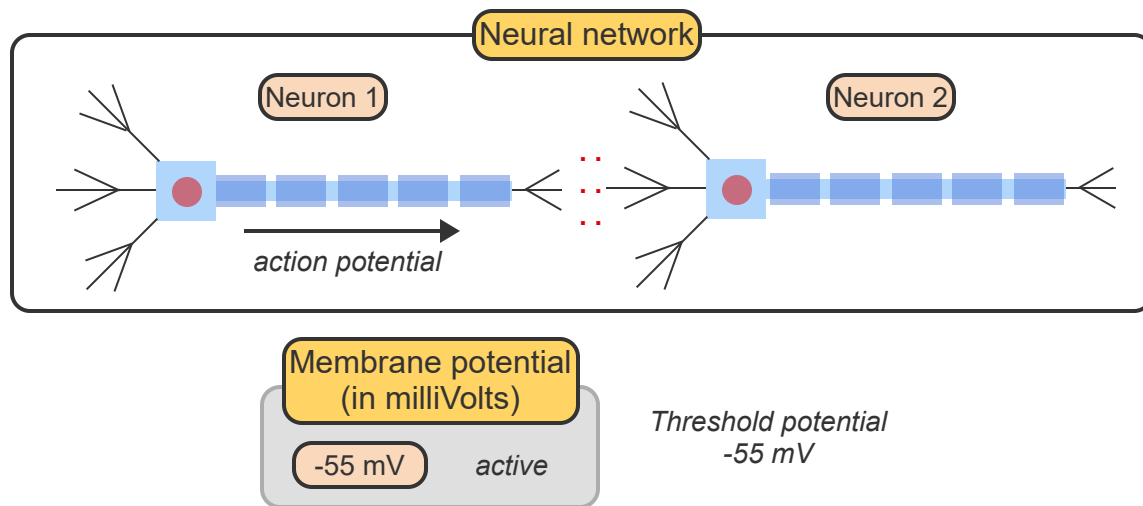
Neurons communicate by sending the following electrical and chemical signals:

- An **action potential** is an electrical signal that travels through a single neuron.
- A **neurotransmitter** is a chemical released at the axon terminals of one neuron to the receptors of another neuron.

When a body encounters stimulus, the electric potential around the neuron's cell membrane increases. The **threshold potential** is the electric potential needed to initiate an action potential. Once the threshold potential is reached, an action potential travels away from the cell body through the axon. At the axon terminals, the action potential is converted to neurotransmitters, which are released to another neuron.

**PARTICIPATION
ACTIVITY**

1.2.3: How neurons send electrical and chemical signals.



Animation content:

Action potentials are generated when inputs from the dendrite cause the membrane potential to reach a threshold potential at around -55 millivolts. Once this threshold potential is reached, an action potential travels down the axon to the axon terminals and neurotransmitters travel from one neuron to the next.

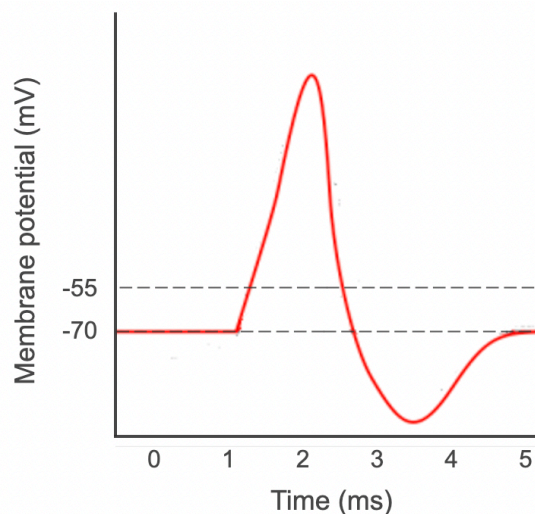
Animation captions:

1. Neurons are nerve cells that transmit electrical and chemical signals. Interconnected neurons are called a neural network.
2. The membrane potential is -70 millivolts (mV) when a neuron is not active. The neuron does not generate action potential until the membrane potential reaches the threshold.
3. When the membrane potential reaches the threshold of -55 mV, the neuron generates an action potential that travels down to the axon terminals.
4. The action potential causes a release of neurotransmitters from the axon terminals to the receptors in another neuron.
5. The dendrites in another neuron convert the neurotransmitters into action potential.

**PARTICIPATION
ACTIVITY**

1.2.4: Action potentials and neurotransmitters.

- 1) The chemical that transmits information between neurons is called ____.
- ☐ a neurotransmitter
- ☐ an action potential
- 2) The neuron with a membrane potential of -60 mV is ____.
- ☐ active
- ☐ not active
- 3) The graph below shows the membrane potential of a neuron from $t = 0$ to $t = 5$ milliseconds. The neuron is stimulated at time $t =$ ____.



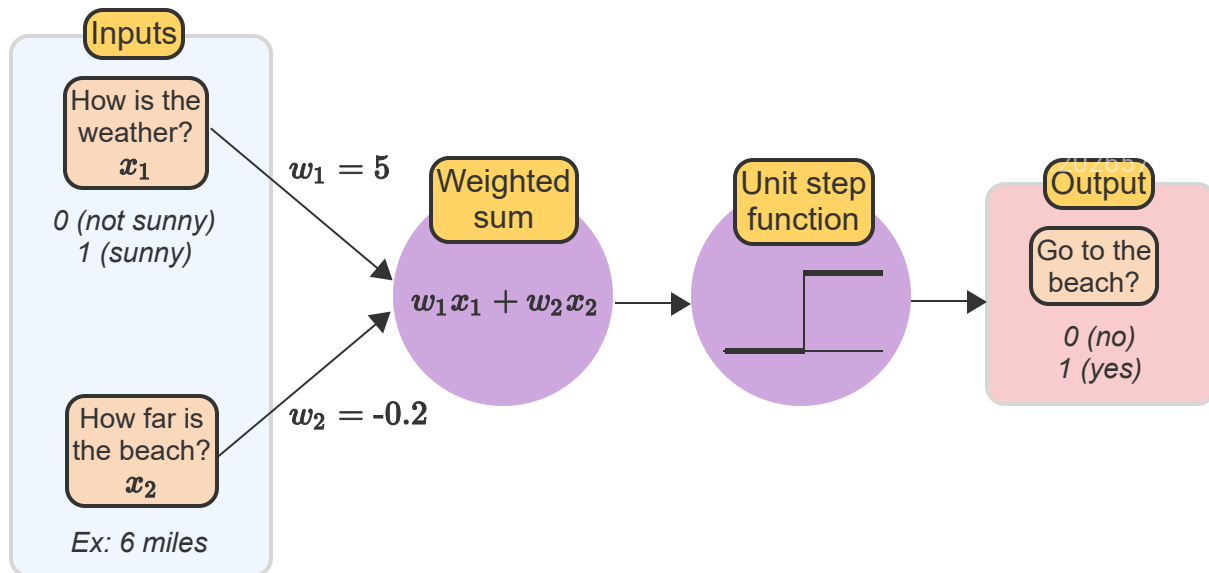
- ☐ 0 ms
- ☐ 1 ms
- ☐ 2 ms

How does an artificial neuron work?

Similar to a biological neuron, an artificial neuron receives inputs and produces an output. The weighted sum is obtained from the inputs and a threshold function such as the unit step function compares the weighted sum to a threshold. The unit step function, for example, produces an output of 1 if the weighted sum is greater than or equal to the threshold and 0 otherwise.

PARTICIPATION
ACTIVITY

1.2.5: Determining whether to go to the beach.

**Animation content:**

An artificial neuron takes in inputs x_1 and x_2 where x_1 is the weather (0 for not sunny and 1 for sunny) and x_2 is the distance from the beach. The weighted sum is $5x_1 - 0.2x_2$. Using a unit step function, if the weighted sum is greater than or equal to the threshold, the neuron fires giving an output of 1. If the weighted sum is less than the threshold, the output is 0.

Animation captions:

1. An artificial neuron takes in one or more inputs. Here, the inputs are weather, x_1 , and distance to the beach, x_2 .
2. The weighted sum is the sum product of the feature's value and the corresponding weight. Here, the weighted sum is $w_1x_1 + w_2x_2 = 5x_1 - 0.2x_2$.
3. Using a unit step function, the neuron gives an output of 1 when the weighted sum is greater than or equal to the threshold and 0 otherwise.

PARTICIPATION
ACTIVITY

1.2.6: Determining whether to go to the beach.

An artificial neuron is used to determine whether a person should go to the beach. This neuron has two inputs with corresponding weights:

- Weather $x_1, w_1 = 5$
- Distance $x_2, w_2 = -0.2$

Using a unit step function with a threshold of 4, the output is 1 (yes) if the weighted sum is greater than or equal to the threshold and 0 (no) otherwise.

The weather today is sunny and the person is 5 miles from the beach.

1) The value of x_1 is ____.

[Check](#)[Show answer](#)

2) The value of x_2 is ____.

[Check](#)[Show answer](#)

3) The weighted sum is ____.

[Check](#)[Show answer](#)

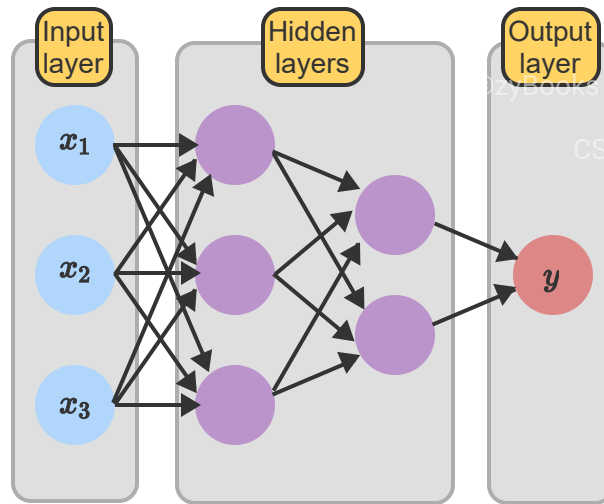
4) The output of the neuron is ____.

[Check](#)[Show answer](#)

What is an artificial neural network?

An **artificial neural network** or **ANN** is a collection of neurons or nodes arranged in multiple layers. The layers of an artificial neural network are:

- The **input layer** takes data from different input features or features to other layers.
- A **hidden layer** assigns weights to inputs and produces numerical outputs from a function.
- The **output layer** produces an output such as a class or an alphanumeric character.

**Animation content:**

A neural network has three layers: an input layer, hidden layer, and output layer. A neural network can have more than one hidden layer. The output layer displays the outcome based on the output of the functions in the hidden layer.

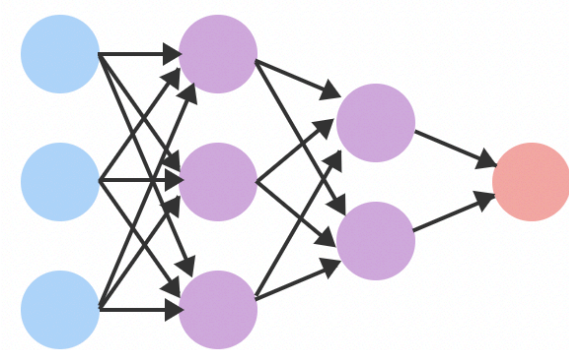
Animation captions:

1. Neurons take values from the input layer and pass these values to the hidden layer without performing any computations.
2. The hidden layers use one or multiple functions using the input features to produce a set of outputs, which are transferred to the output layer.
3. The output layer displays the outcome of a machine-learning task based on functions in the hidden layers.

1) Which of the following layers are exposed to the features of a dataset?

- ☐ Input layer
- ☐ Hidden layers
- ☐ Output layer

2) How many neurons are in the hidden layers of the artificial neural network shown below?



- ☐ 1
- ☐ 3
- ☐ 5

3) Can an artificial neural network have more than one node in the output layer?

- ☐ No
- ☐ Yes

CHALLENGE ACTIVITY

1.2.1: Artificial neural networks.

758618.6080406.qx3zqy7

Start

A neuron has a threshold potential of -55 mV.

The neuron is active when the membrane potential reaches .

Once the neuron is active, which of the following events occur? Check all that apply.

☐ A stimulus is encountered.

- ☐ The membrane potential begins to increase.
- ☐ Neurotransmitters are released from the axon terminals.

1

2

Check

Next

Alternative text not found

1.3 Single-layer perceptron

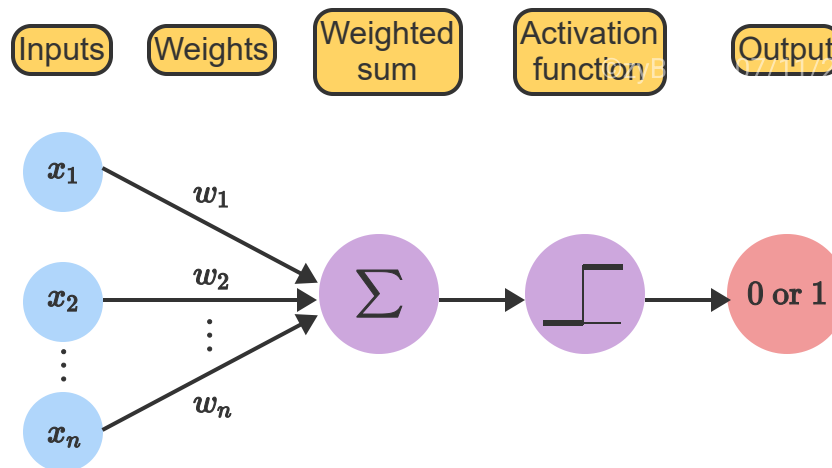
Learn to:

- Identify parts of a perceptron.
- Use perceptron models to classify instances.
- Use bias terms and activation functions to modify a perceptron.
- Explain the steps of training a perceptron.
- Implement a perceptron model using `scikit-learn`.

What is a single-layer perceptron?

A **single-layer perceptron** is a neural network used for binary classification. A **binary classifier** groups data into one of two classes. Ex: Determining whether to go to the beach based on weather conditions is a binary classification problem, because the answer can be only yes or no.

A perceptron has five parts: inputs, weights, weighted sum, activation function, and output. The inputs are fed directly to the outputs via a series of weights and an activation function. The phrase single-layer refers to the one layer of links between input and output, not the number of hidden layers. In this section, single-layer perceptrons will be referred to as perceptrons.

**Animation content:**

Inputs $x_1, x_2, \dots, x_n$ with corresponding weights $w_1, w_2, \dots, w_n$ are used to find the weighted sum. The weighted sum is used by an activation function to give an output of either 0 or 1.

Animation captions:

1. Inputs are numerical values that are taken by a perceptron.
2. Each input is assigned a weight, which is a scalar value and corresponds to how important the input is to the model.
3. The weighted sum is the sum product of the inputs and their corresponding weights:

$$w_1x_1 + w_2x_2 + \dots, w_nx_n.$$
4. An activation function, such as the unit step function, uses the weighted sum to give output values that fall within a certain range.
5. Since a perceptron is a binary classifier, the output has two possible values: 0 or 1.

1) What is a weighted sum?

- ☐ The sum of all inputs
- ☐ The sum of the products of inputs and corresponding weights
- ☐ The sum of all weights

2) Which of the following tasks can a perceptron perform?

- ☐ Predicting the selling price of a house
- ☐ Classifying a house as sold, active, or pending
- ☐ Determining whether a house is close to a particular school

3) How many hidden layers does a perceptron contain?

- ☐ 0
- ☐ 1
- ☐ 2

How does a perceptron classify instances?

The weighted sum

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n$$

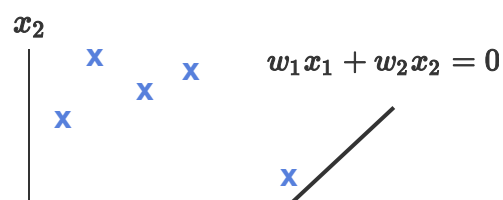
is a hyperplane that divides the training data into two parts.

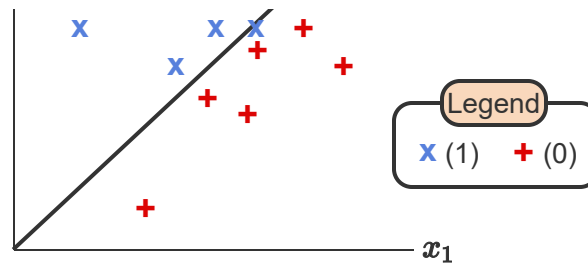
An instance on or above $z = 0$ is classified into one class, while an instance below $z = 0$ is classified into another class.

PARTICIPATION ACTIVITY

1.3.3: Interpretation of the weighted sum.

©zyBooks 07/11/26 03:10 EDT 304020





Animation content:

Instances correspond to a point on the graph. Instances on or above a line are classified as class 1 and instances below a line are classified as class 0.

Animation captions:

1. A perceptron can classify instances based on the values of the input features. Here, the training data contains two features x_1 and x_2 .
2. The weighted sum $z = w_1x_1 + w_2x_2 + \dots w_nx_n = 0$ is a hyperplane. Here, $z = w_1x_1 + w_2x_2 = 0$ is a line passing through the origin.
3. Instances on or above the line are classified into one class (1), while instances below are classified into another class (0).

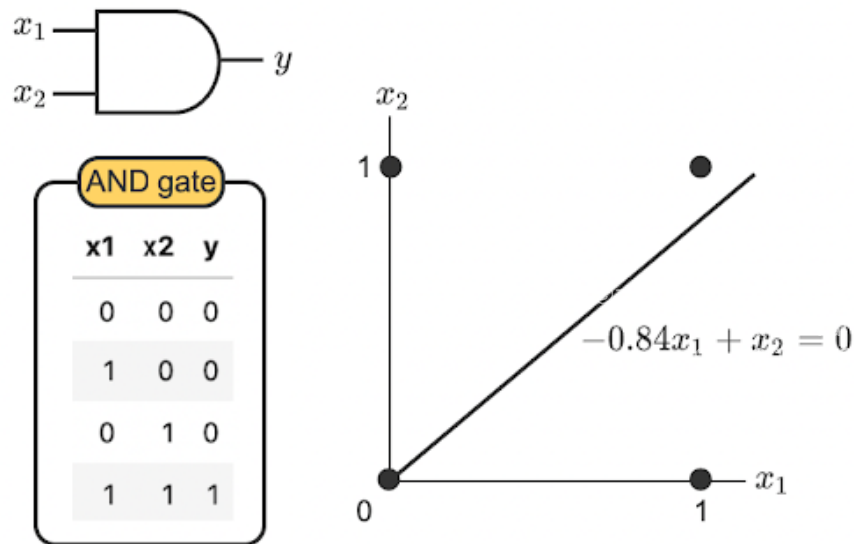
PARTICIPATION ACTIVITY

1.3.4: Perceptron model for an AND gate.



A logic gate is a simplified computational model with binary inputs and outputs. An AND gate is a logic gate that outputs 1 when both inputs are 1 and outputs 0 otherwise. AND gates have a variety of applications in digital electronics and data transmission.

In the figure below, a perceptron model has a weighted sum of $-0.84x_1 + x_2$, which is represented by the line $-0.84x_1 + x_2 = 0$.



- 1) The weight of x_1 is ____.
 - ☐ 1
 - ☐ 0
 - ☐ -0.84
- 2) An instance with input values $x_1 = 1$ and $x_2 = 1$ will be classified by the perceptron as ____.
 - ☐ 0
 - ☐ 1
- 3) The actual classification of an instance with input values $x_1 = 1$ and $x_2 = 1$ is ____.
 - ☐ 0
 - ☐ 1
- 4) An instance with input values $x_1 = 0$ and $x_2 = 1$ will be classified by the perceptron as ____.
 - ☐ 0
 - ☐ 1

5) The actual classification of an instance with input values $x_1 = 0$ and $x_2 = 1$ is _____.

- ☐ 0
- ☐ 1

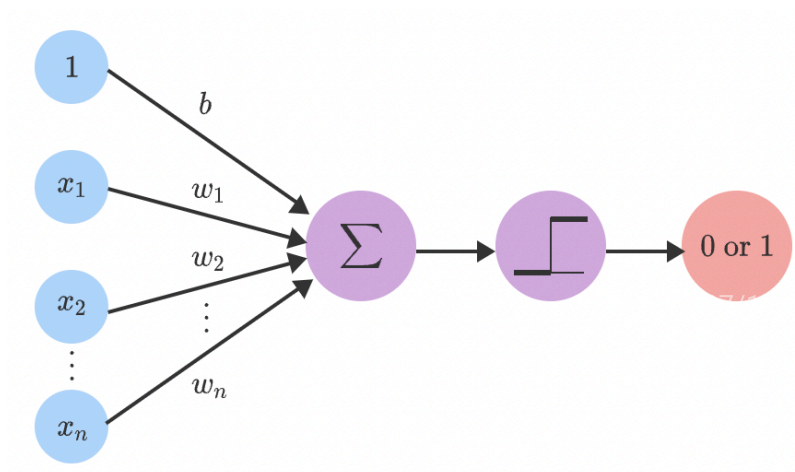
6) Does the perceptron model perform well?

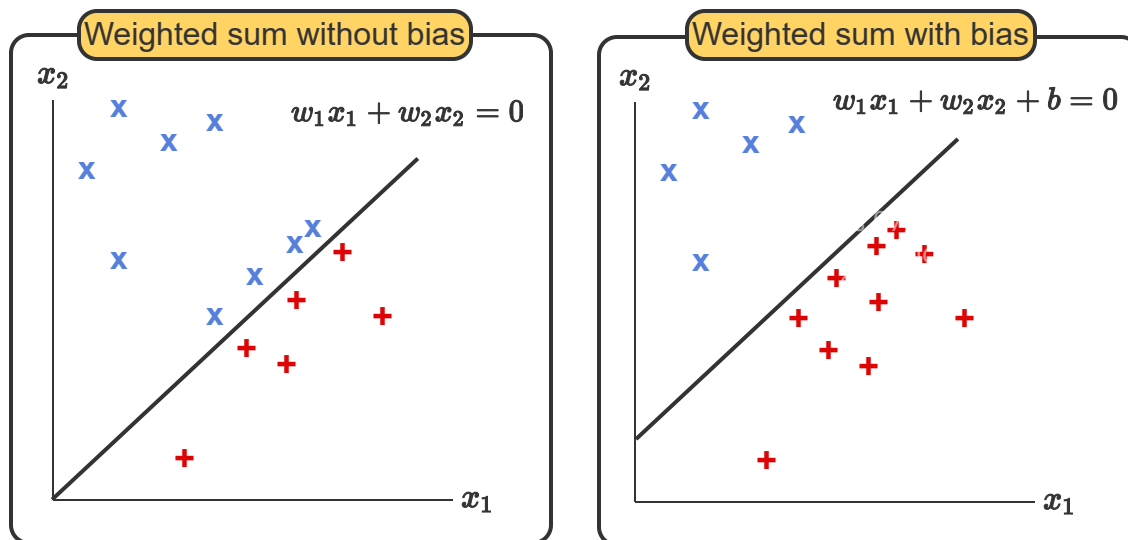
- ☐ Yes, because the model correctly classified an instance with $x_1 = 1$ and $x_2 = 1$.
- ☐ No, because the model incorrectly classified an instance with $x_1 = 0$ and $x_2 = 1$.
- ☐ No, because the model incorrectly classifies 2 out of 4 instances in the AND gate.

What is bias?

Bias is a term added to the weighted sum to improve the perceptron's accuracy. A perceptron model that includes a bias term b contains an additional input feature with a constant value of 1 and weight b .

Figure 1.3.1: Perceptron that has a bias term.





Animation content:

7 points are near each other. 4 points are slightly above the line and 3 points are slightly below the line. Adding a bias term moves the line, so that all 7 points are below the line. The bias term makes the model fit the data better.

Animation captions:

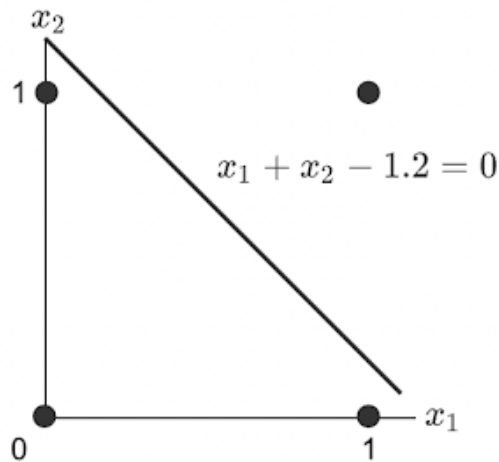
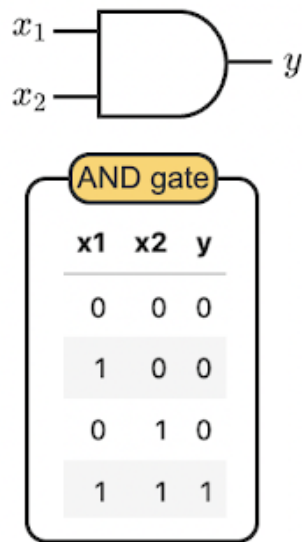
1. In this dataset, four instances could potentially be classified into class 0 instead of class 1.
2. Adding a bias term moves the line up, which appears to fit the training data better.
3. Including a bias term changes the classification of four instances from class 1 to class 0.

PARTICIPATION ACTIVITY

1.3.6: Perceptron model with a bias term for an AND gate.



A perceptron model with a bias term has a weighted sum of $\mathbf{x_1 + x_2 - 1.2}$ represented by the line $\mathbf{x_1 + x_2 - 1.2 = 0}$ in the figure below.



- 1) The weight of the bias term is ____.
- 2) An instance with input values $x_1 = 1$ and $x_2 = 1$ will be classified by the perceptron as ____.
- 3) The actual classification of an instance with input values $x_1 = 1$ and $x_2 = 1$ is ____.
- 4) An instance with input values $x_1 = 0$ and $x_2 = 1$ will be classified by the perceptron as ____.

5) The actual classification of an instance with input values $x_1 = 0$ and $x_2 = 1$ is

_____.

- ☐ 0
- ☐ 1

6) Does the perceptron model with a bias term perform well?

- ☐ Yes, because the model correctly classifies an instance with $x_1 = 1$ and $x_2 = 1$.
- ☐ Yes, because the model correctly classifies all instances.
- ☐ No, because the model was only used on training data.

What is an activation function?

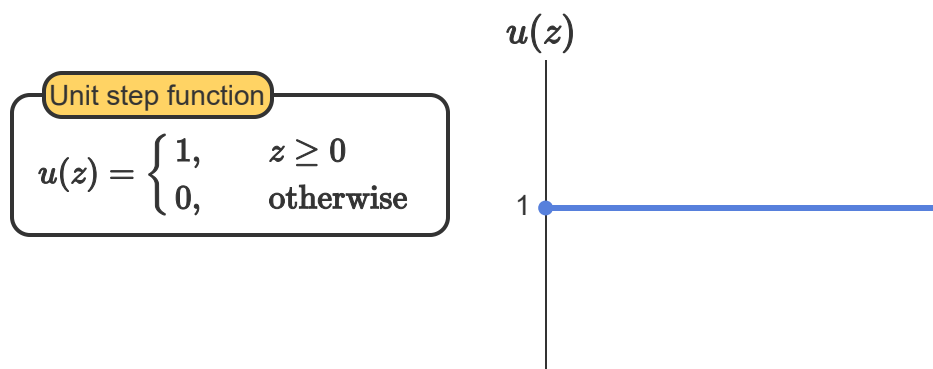
An **activation function** gives the output for a perceptron. The most common activation function is the unit step function. The unit step function $u(z)$ with a bias term b is defined as:

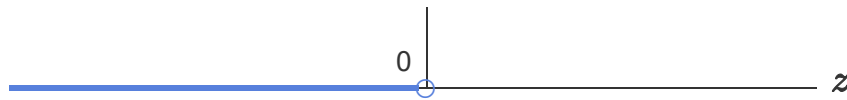
$$u(z) = \begin{cases} 1, & z \geq 0 \\ 0, & \text{otherwise} \end{cases}$$

where $z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$.

PARTICIPATION ACTIVITY

1.3.7: Unit step function.





Animation content:

The unit step function is a function that gives an output of 1 when the weighted sum z is greater than or equal to 0, and gives an output of 0 when the weighted sum z is less than 0.

Animation captions:

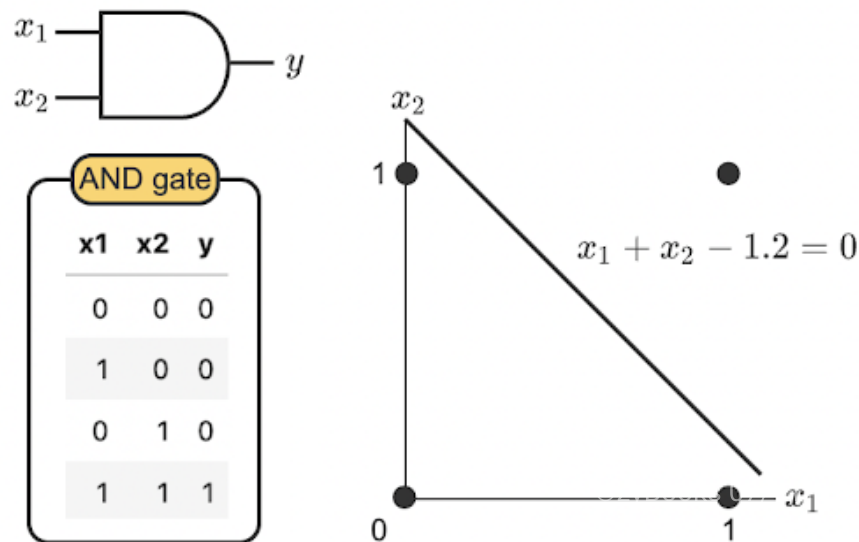
1. The unit step function $u(z)$ is a common activation function.
2. When the weighted sum z is greater than or equal to zero, the output of $u(z)$ is 1.
3. When the weighted sum z is less than 0, the output of $u(z)$ is 0.

PARTICIPATION ACTIVITY

1.3.8: Applying the unit step function to a weighted sum for an AND gate.



A perceptron model with a bias term has a weighted sum of $x_1 + x_2 - 1.2$ and is represented by the line $x_1 + x_2 - 1.2 = 0$ in the figure below.



- 1) The weighted sum for an instance with input values $x_1 = 1$ and $x_2 = 1$ is _____.

**Check**[Show answer](#)

- 2) The output of the unit step function for an instance with input values $x_1 = 1$ and $x_2 = 1$ is _____.

**Check**[Show answer](#)

What does training a perceptron mean?

Training a neural network involves finding the weights that minimize the average error in the training data. The algorithm begins with initial weights and iteratively updates those weights using the rule:

$$\text{weight} = \text{weight} + \text{learning rate} * \text{error} * \text{input}$$

where

$$\text{error} = \text{actual class} - \text{predicted class}.$$

An **epoch** is an iteration of weight updates over all instances in the training data. Smaller learning rates need more epochs to train a neural network, while larger learning rates need less epochs to train but may give less optimal weights.

Algorithm 1.3.1: Algorithm for training a perceptron.

Step 1: Set initial weights in the model.

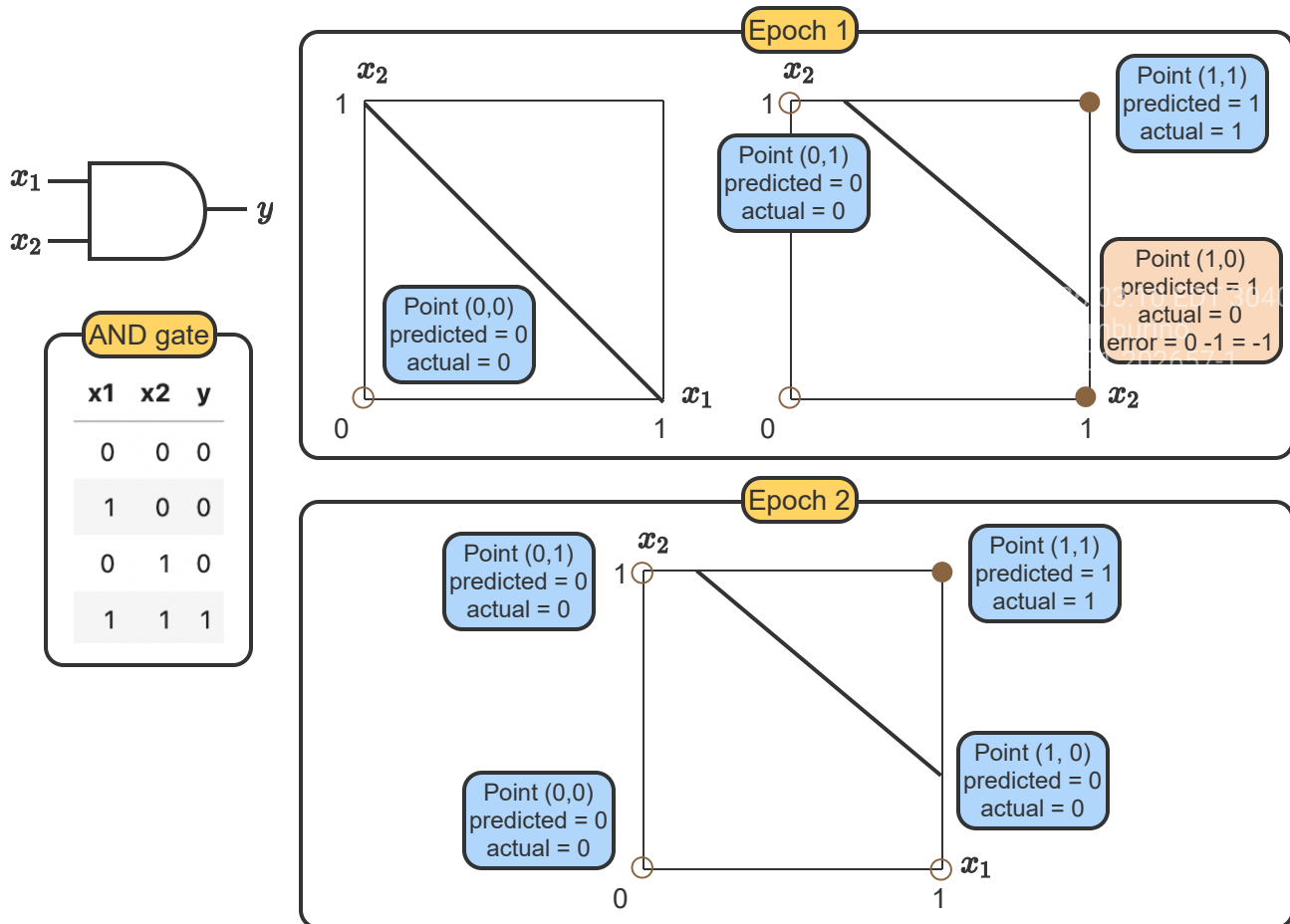
Step 2: Update weights based on learning rate and errors.

Step 3: Repeat steps 1 and 2 until the maximum number of epochs or minimum error threshold is reached.

**PARTICIPATION
ACTIVITY**

1.3.9: Training a perceptron model for an AND gate.





Animation content:

Step 1

The first step in the algorithm is to initialize the weights. Here, the initial weights are $w_1 = 1$, $w_2 = 1$, and $b = -1$, which represent a line with slope $-w_1 / w_2 = -1$ and intercept of $-b / w_2 = 1$.

Step 2

Point (0,0) is below the line, so the model correctly classifies this point as class 0.

No weight updates are needed.

Step 3

Point (1,0) is on the line, so the model incorrectly classifies this point as class 1.

Using a learning rate of $r = 0.1$, the updated weights are $w_1 = 0.9$, $w_2 = 1$, and $b = -1.1$.

Step 4

Points (0,1) and (1,1) are correctly classified, so no further weight updates are needed.

After epoch 1, the weights are $w_1 = 0.9$, $w_2 = 1$, and $b = -1.1$.

Step 5

No updates are needed in epoch 2, because all instances are classified correctly.

Instances below the line are classified as 0 and the instance on or above the line is classified as 1.

Animation captions:

1. The first step in the algorithm is to initialize the weights. Here, the initial weights are $w_1 = 1$, $w_2 = 1$, and $b = -1$, which represent a line with slope $-w_1 / w_2 = -1$ and intercept of $-b / w_2 = 1$.
2. Point (0,0) is below the line, so the model correctly classifies this point as class 0. No weight updates are needed.
3. Point (1,0) is on the line, so the model incorrectly classifies this point as class 1. Using a learning rate of $r = 0.1$, the updated weights are $w_1 = 0.9$, $w_2 = 1$, and $b = -1.1$.
4. Points (0,1) and (1,1) are correctly classified, so no further weight updates are needed. After epoch 1, the weights are $w_1 = 0.9$, $w_2 = 1$, and $b = -1.1$.
5. No updates are needed in epoch 2, because all instances are classified correctly. Instances below the line are classified as 0, and the instance on or above the line is classified as 1.

PARTICIPATION ACTIVITY

1.3.10: Training a perceptron model for an AND gate.

The initial weights of a perceptron model for an AND gate are $w_1 = 0.7$, $w_2 = 0.4$, and $b = 0.7$. The weights at the end of epoch 3 are $w_1 = 0.4$, $w_2 = 0.1$, and $b = -0.2$.

Find the weights at the end of epoch 4 using a learning rate of $r = 0.1$. At the beginning of epoch 4, only the instance where $x_1 = 1$ and $x_2 = 0$ is incorrectly classified. The predicted class for this instance is 1, but the actual class is 0.

1) At the end of epoch 4, $w_1 =$ ____.



Check

Show answer

2) At the end of epoch 4, $w_2 =$ ____.



Check

Show answer

3) At the end of epoch 4, $b =$ ____.



Check

Show answer

Creating a perceptron model in Python

`Perceptron()` creates a perceptron model. The most common parameters are the number of epochs, `max_iter`, and the learning rate `eta0`. The rest of the parameters and attributes can be found in the [Perceptron documentation](#).

Table 1.3.1: Common attributes of a perceptron model.

Attribute	Description
<code>coef_</code>	Weights assigned to the input features
<code>intercept_</code>	Weight assigned to the bias term
<code>max_iter</code>	Number of epochs to reach the stopping criterion

Try 1.3.1: Perceptron models in Python.

[Full screen](#)

The [Haberman's Survival Dataset](#) contains cases from a study that was conducted between 1958 and 1970 at the University of Chicago's Billings Hospital on the survival of patients who had undergone surgery for breast cancer. The dataset contains four features: the patient's age, year of surgery, number of cancer nodes detected, and whether the patient survived after 5 years.

The Python code below creates a perceptron model that predicts whether a patient survived based on age, year of surgery, and number of cancer nodes detected.

- Click the double-right arrow to restart the kernel and run all cells.
- Examine the code below.
- Change the learning rate from 0.1 to 0.05. Explore any changes in the output.

[History](#)

Start your Python (Jupyter Notebook) environment



Model Solution [^](#)

**PARTICIPATION
ACTIVITY**

1.3.11: Creating a perceptron model in Python.

- 1) What is the code to initialize a perceptron model `clf` with a maximum of 2,500 epochs and a learning rate of 0.2?

Check[Show answer](#)

- 2) What is the code to fit a perceptron model `clf` to a training set `x_train` and a test set `y_train`?

Check[Show answer](#)

- 3) What is the code to display the weights given a fitted perceptron model `clf`?

Check[Show answer](#)**CHALLENGE
ACTIVITY**

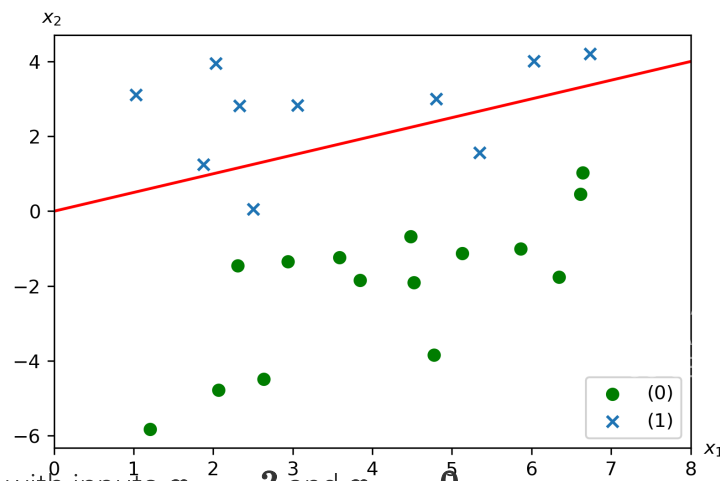
1.3.1: Single-layer perceptron.

758618.6080406.qx3zqy7

Start

A perceptron uses the weighted sum $z = -0.5x_1 + x_2$, represented by the line $z = -0.5x_1 + x_2 = 0$ on the following graph.

The instances represented by blue x's are classified as 1, and the instances represented by green circles are classified as 0.



A new instance is added with inputs $x_1 = 3$ and $x_2 = 0$.

What is the weighted sum for this instance?

How does the perceptron classify this instance?

1	2	3
---	---	---

Alternative text not found

CHALLENGE ACTIVITY

1.3.2: Single-layer perceptron using scikit-learn.



758618.6080406.qx3zqy7

This dataset contains 13 health-related attributes from 100 patients and one attribute denoting whether or not the patient has heart disease.

- Initialize a perceptron model `percModel` with a maximum of 2000 epochs and a learning rate of 0.1.

The code contains all imports, loads the dataset, splits the dataset into test and train data, fits the model, and prints the weights of the fitted perceptron model.


```
1 # Import packages and functions
2 import pandas as pd
3 import numpy as np
4 from sklearn.linear_model import Perceptron
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
```

```
7
8 # Load the dataset
9 heart = pd.read_csv('heart.csv')
10
11 # Slices the features of the dataset
12 X = heart[['trestbps', 'age', 'thalach']]
13 y = heart[['target']]
14
15 # Scales the features
16 scaler = StandardScaler()
17 XScaled = pd.DataFrame(scaler.fit_transform(X), columns=['trestbps', 'age', 'thalach'])
18
19 # Splits the data into train and test sets
20 XTrain, XTest, yTrain, yTest = train_test_split(XScaled, y, test_size=0.2, random_s
21
22 # Initializes and fits a perceptron model
23 percModel = # Your code goes here
24 percModel.fit(XTrain, np.ravel(yTrain))
25
26 print(percModel.coef_)
27 print(percModel.intercept_)
```

1

2

3

Check

Next level

1.4 Nonlinear activation functions

Learn to:

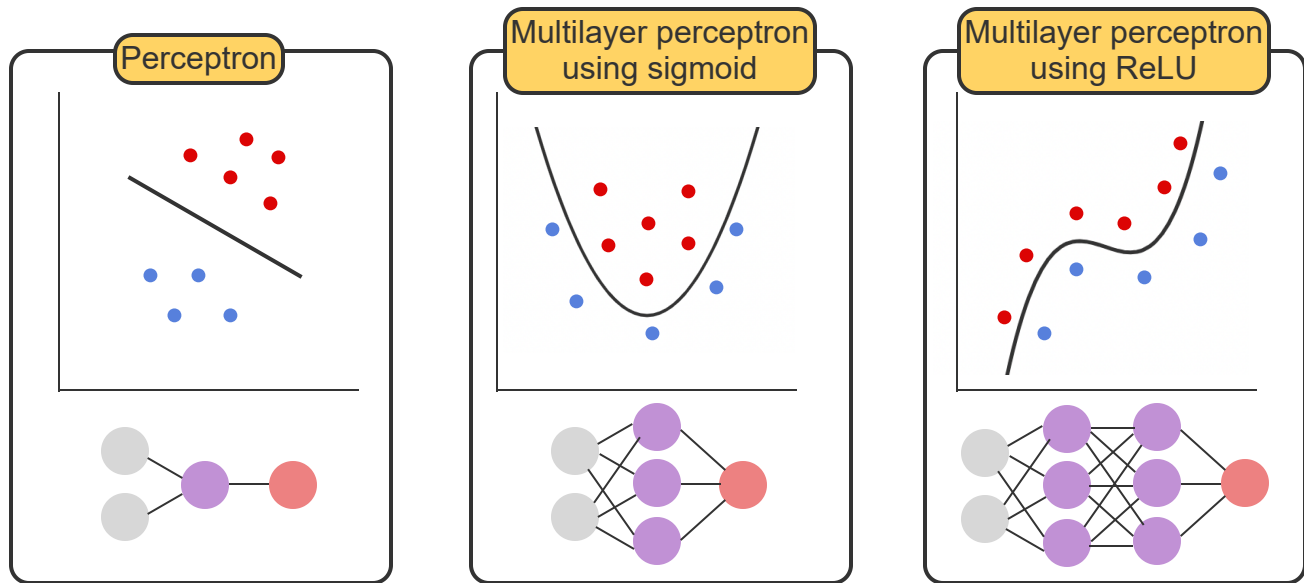
- Define nonlinear activation functions: sigmoid, hyperbolic tangent, ReLU, and softmax.
- Explain the purpose of nonlinear activation functions.
- Explain when each activation function should be used.

Why are nonlinear activation functions needed?

A perceptron is useful for finding a line or hyperplane as a decision boundary to classify an instance into one of two groups. However, a nonlinear decision boundary might be better in some situations. For more complicated data, a neural network with multiple hidden layers and nonlinear activation functions might be needed for more accurate predictions.

PARTICIPATION ACTIVITY

1.4.1: Why use nonlinear activation functions?



Animation content:

A perceptron can divide a feature space using a line. A multilayer perceptron can divide more complicated feature spaces. Increasing the number of layers and using other types of activation functions allows the division of more complicated feature spaces for better model performance.

Animation captions:

1. Perceptron neural networks classify data using a hyperplane as a decision boundary. Here, the decision boundary is a line, so updating the weights and bias changes the slope and intercept.
2. Certain datasets cannot be classified using a linear decision boundary. nonlinear activation functions and more neurons in the hidden layer can improve model performance.
3. More complicated datasets require multiple hidden layers and other types of activation functions, such as the rectified linear unit (ReLU).

**PARTICIPATION
ACTIVITY**

1) A perceptron uses a line for binary classification of an instance.



- ☐ True
- ☐ False

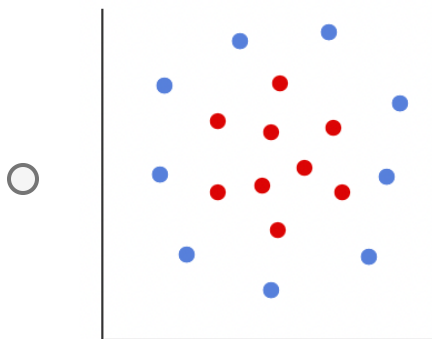
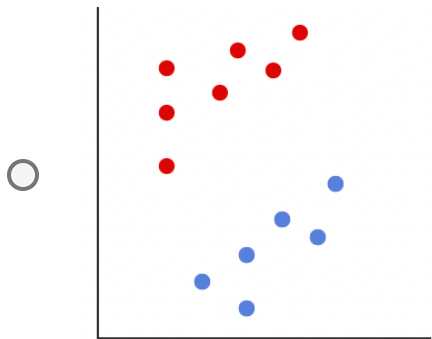
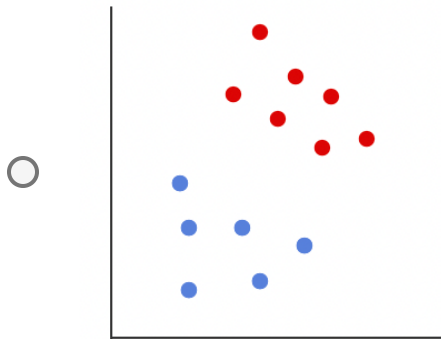
2) A multilayered perceptron with nonlinear activation functions is used instead of a single-layer perceptron to ____.



- ☐ form linear decision boundaries
- ☐ help the weights converge faster
- ☐ form nonlinear decision boundaries



- 3) For which dataset should a multilayered perceptron with nonlinear activation functions be used?



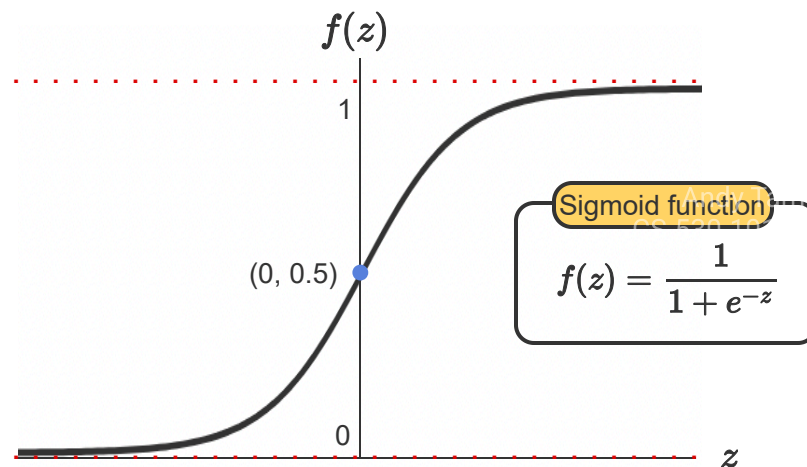
Sigmoid

A **sigmoid** is a function that takes in any real value as input and returns a real number between 0 and 1 as output. Because the value of a probability is between 0 and 1, a sigmoid function is often used for predicting a probability as output.

The formula for a sigmoid is:

$$f(z) = \frac{1}{1 + e^{-z}}.$$





Animation content:

A sigmoid function has the equation $f(z) = 1/(1+e^{-z})$ and has an s-shaped graph. The y-intercept is located at (0,0.5) and the horizontal asymptotes are at $y = 1$ and $y = 0$.

Animation captions:

1. A sigmoid activation function is useful when the neural network deals with nonlinear decision boundaries.
2. The domain of the sigmoid function is all real numbers. The range of the sigmoid function is **(0, 1)**.
3. The neuron activates when $z \geq 0$, but this threshold can be set arbitrarily depending on the learning task.

PARTICIPATION ACTIVITY

1.4.4: Sigmoid.

- 1) The value of the sigmoid function at $z = 0$ is ____.

Check

Show answer

- 2) The sigmoid function approaches _____
as z approaches ∞ .



Check

Show answer

- 3) The sigmoid function approaches _____
as z approaches $-\infty$.



Check

Show answer

Hyperbolic tangent

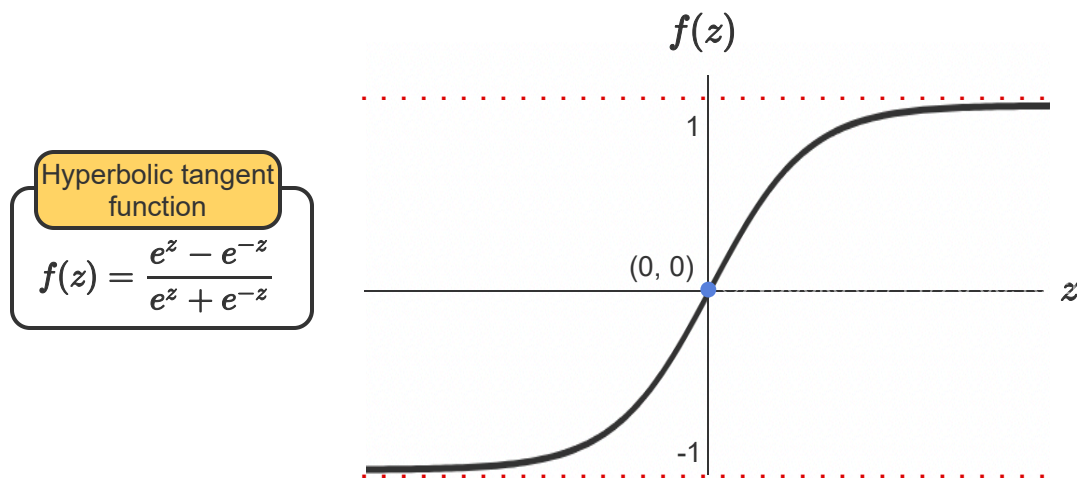
A **hyperbolic tangent** is a function that takes in any real value as input and returns a real number between -1 and 1 as output.

The formula for a hyperbolic tangent is:

$$f(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}.$$

PARTICIPATION ACTIVITY

1.4.5: Hyperbolic tangent.



Animation content:

A hyperbolic tangent function has the equation $f(z) = (e^z - e^{-z}) / (e^z + e^{-z})$ and has an s-shaped graph. The y-intercept is located at (0,0) and the horizontal asymptotes are at $y = 1$ and $y = -1$.

Animation captions:

1. A hyperbolic tangent activation function is another nonlinear activation function.
2. The domain of the hyperbolic tangent function is all real numbers. The range of the hyperbolic tangent function is $(-1, 1)$.
3. The neuron activates when $z \geq 0$, but this threshold can be set arbitrarily depending on the learning task.

PARTICIPATION ACTIVITY

1.4.6: Hyperbolic tangent.

- 1) The value of the hyperbolic tangent function at $z = 0$ is ____.

Check

Show answer

- 2) The hyperbolic tangent function approaches ____ as z approaches ∞ .

Check

Show answer

- 3) The hyperbolic tangent function approaches ____ as z approaches $-\infty$.

Check

Show answer

Rectified linear unit

The **rectified linear unit**, or **ReLU**, is a function that takes in any real value as input and returns either 0 or the input, whichever is greater.

The formula for a ReLU is:

$$f(z) = \max\{0, z\}.$$

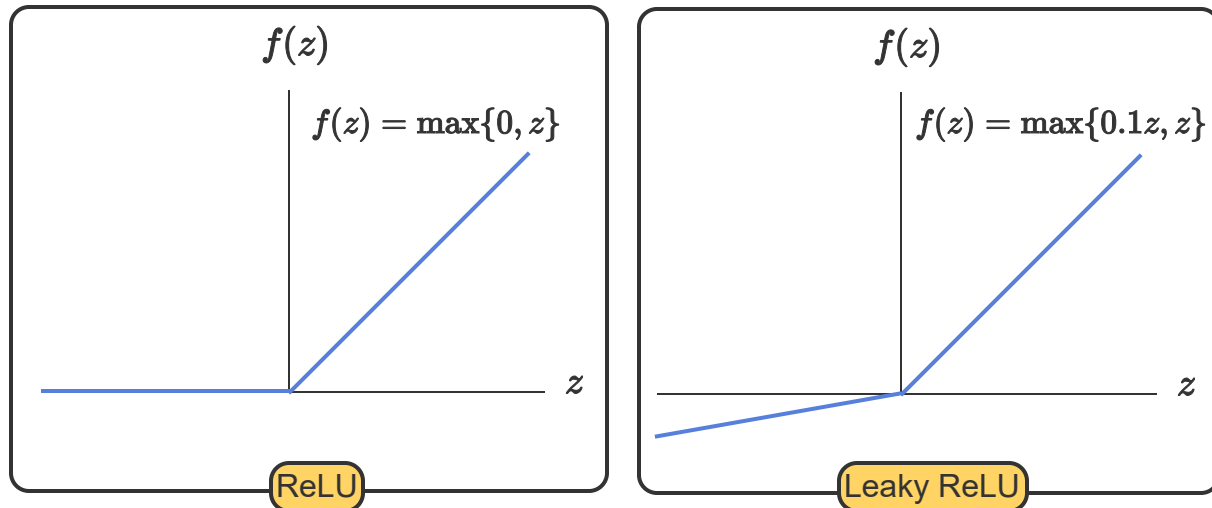
The **leaky ReLU** is a function with a small slope when the input value is negative.

The formula for a leaky ReLU is:

$$f(z) = \max\{0.1z, z\}.$$

PARTICIPATION ACTIVITY

1.4.7: ReLU and leaky ReLU.



Animation content:

The ReLU function has an equation $f(z) = \max\{0, z\}$, which means that the function has a value of $f(z) = 0$ when z is less than 0 and $f(z) = z$ when z is greater than or equal to 0. The leaky ReLU function has an equation $f(z) = \max\{0.1z, z\}$, which means that the function has a value of $f(z) = 0.1z$ when z is less than 0 and $f(z) = z$ when z is greater than or equal to 0.

Animation captions:

1. The output of a rectified linear function is the maximum of 0 and x . In other words, the function gives an output of 0 when $z < 0$ and z when $z \geq 0$.

2. The output of a leaky rectified linear function is the maximum of $0.1z$ and z . In other words, the function gives an output of $0.1z$ when $z < 0$ and z when $z \geq 0$.

**PARTICIPATION
ACTIVITY**

1.4.8: ReLU and leaky ReLU.



- 1) The value of the ReLU function at $z = -10$ is ____.

**Check**[Show answer](#)

- 2) The value of the leaky ReLU function at $z = -10$ is ____.

**Check**[Show answer](#)

- 3) The value of the ReLU function at $z = 10$ is ____.

**Check**[Show answer](#)

- 4) The value of the leaky ReLU function at $z = 10$ is ____.

**Check**[Show answer](#)

Softmax

A **softmax** function takes in a vector and returns a vector with the same number of components as output. The formula for a softmax function is:

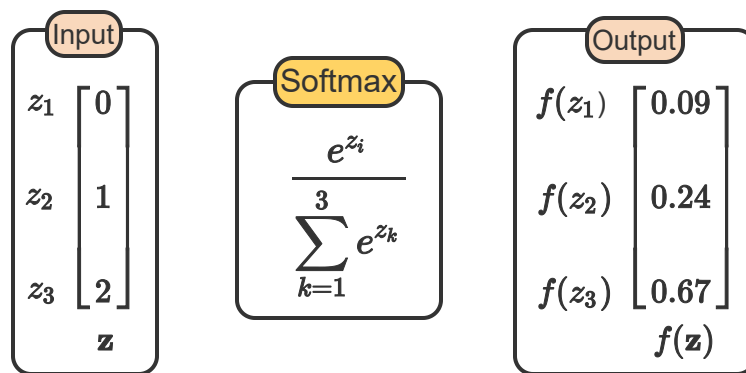
$$f(z_j) = \frac{e^{z_j}}{\sum_{k=1}^n e^{z_k}}$$

for $j = 1, \dots, n$.

A softmax function is useful when making multiclass predictions. A neural network used for multiclass predictions has one node in the output layer for each class. The output corresponds to the predicted probabilities of the classes, which add up to 1.

PARTICIPATION ACTIVITY

1.4.9: Using the softmax function.



$$\sum_{k=1}^3 e^{z_k} = e^0 + e^1 + e^2 = 11.11$$

Animation content:

The softmax function takes in a vector and outputs a normalized vector. When the number of inputs is $n = 3$, the vector will have 3 components. If the input vector has components $z_1 = 0$, $z_2 = 1$, and $z_3 = 2$, the first component of the output is $f(z_1) = e^{z_1} / \sum (e^{z_1} + e^{z_2} + e^{z_3}) = 0.09$. The other two components of the output are $f(z_2) = 0.24$ and $f(z_3) = 0.67$.

Animation captions:

1. The softmax function takes in a vector as input. Here, $z_1 = 0$, $z_2 = 1$, and $z_3 = 2$ are the components of vector $\mathbf{z}$.
2. The softmax function uses the sum of the exponentials of each of the inputs.

3. To find the output's first component, the input's first component should be exponentiated and then divided by the sum of the exponentials of the input's components.
4. The rest of the output's components can be calculated similarly. Here, $f(z_1) = 0.09$, $f(z_2) = 0.24$, and $f(z_3) = 0.67$ are the components of vector $f(\mathbf{z})$.

**PARTICIPATION
ACTIVITY**

1.4.10: Softmax.

 zyBooks 07/11/26 03:10 EDT 304020
 Andy Tamburro

1) The output of a softmax function is a ____.

- ☐ vector
- ☐ scalar

2) The softmax function gives an output of ____ when $\mathbf{z} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$.

- ☐ $\begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$
- ☐ $\begin{bmatrix} 1 \\ 1 \end{bmatrix}$
- ☐ $\begin{bmatrix} 0.5 \\ 1 \end{bmatrix}$

3) The components of the output of a softmax function add up to ____.

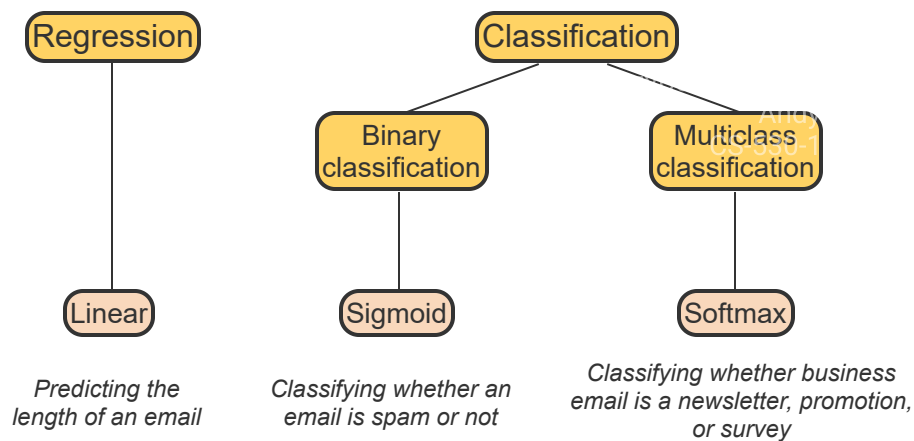
- ☐ greater than 1
- ☐ less than 1
- ☐ 1

Which activation function should be used?

The activation function needed in a neural network depends on whether the machine learning task is performing regression or classification and the type of neural network. For instance, a unit step function is often used with a single-layer perceptron, while a ReLU function is often used with a multilayer perceptron. Networks that involve a time sequence where information cycles through a loop, known as recurrent neural networks, often use the hyperbolic tangent function.

**PARTICIPATION
ACTIVITY**

1.4.11: Determining which activation function to use.

**Animation content:**

Linear activation functions should be used for regression tasks. Sigmoid should be used for binary classifiers. Softmax should be used for multiclass classification problems.

Animation captions:

1. A linear activation function such as $f(z) = z$ is best suited for regression tasks such as predicting the length of an email based on the sender and presence of certain keywords.
2. A sigmoid is best suited for binary classification tasks such as determining whether an email is spam or not. Other functions that work well are unit step and hyperbolic tangent.
3. A softmax function is best suited for multiclass classification tasks such as classifying a business email as a newsletter, promotion, or survey.

**PARTICIPATION
ACTIVITY**

1.4.12: Determining which activation function should be used.

How to use this tool ▼

Sigmoid

Softmax

Linear

Determining whether a single-family home will go under contract in less than two weeks, between two weeks and one month, or longer than one month

Determining whether a single-family home will go under contract above the listing price or below the listing price

Predicting the listing price of a single-family home based on the number of bedrooms and bathrooms, square footage, and distance from various amenities

[Reset](#)**CHALLENGE
ACTIVITY**

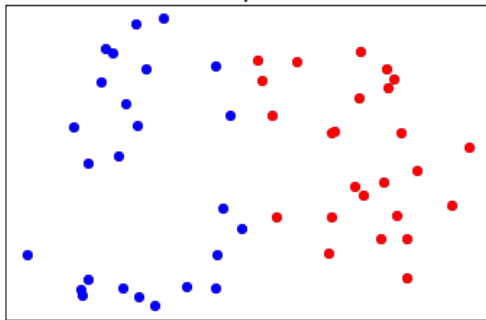
1.4.1: Nonlinear activation functions.



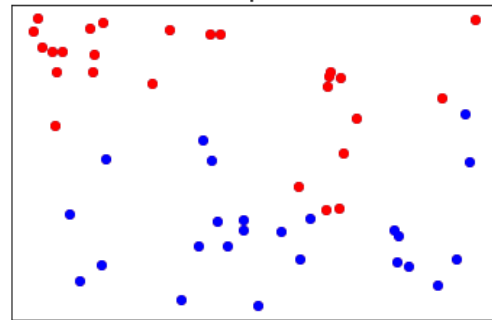
758618.6080406.qx3zqy7

[Start](#)

Graph A



Graph B



For which dataset, should a multilayered perceptron with nonlinear activation function be used?

1

2

3

Check

Next

Alternative text not found

1.5 Multilayer perceptron

Learn to:

- Define feed-forward networks and multilayer perceptrons.
- Explain forward propagation and backward propagation.
- Use loss functions and loss curves to examine a neural network.
- Implement a multilayer perceptron using `scikit-learn`.

What is a multilayer perceptron?

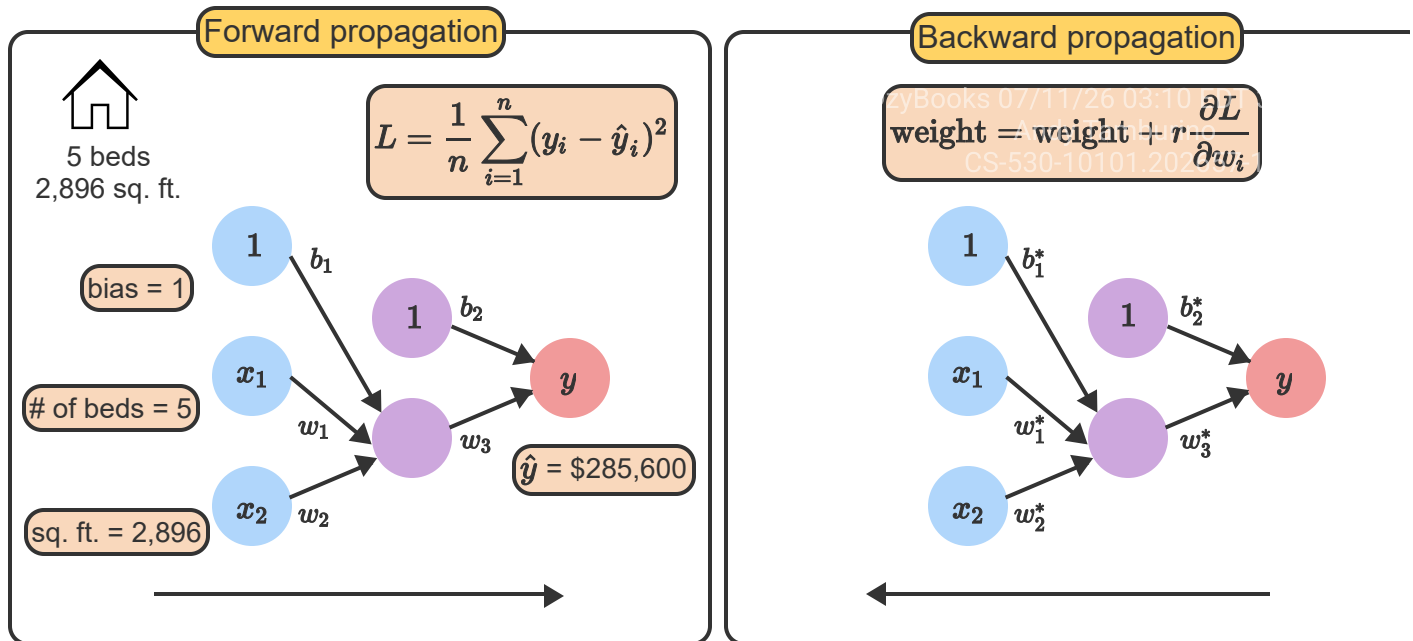
A single-layer perceptron is an example of a feed-forward neural network. A **feed-forward network** is a network where connections between neurons do not form a cycle. A **multilayer perceptron** is a type of feed-forward neural network with at least one hidden layer containing multiple neurons. Unlike single-layer perceptrons that can only be used for classification tasks, multilayer perceptrons can also be used for regression tasks.

Training a multilayer perceptron involves two steps.

- **Forward propagation** is the step where inputs are fed into the neural network, and outputs from neurons are passed in one direction from one layer to each successive layer. In this step, the error is calculated using a loss function.
- **Backward propagation** or **backpropagation** is the process of iteratively adjusting the weights in the network to minimize prediction errors. In this step, the rate of change of the loss function L with respect to each weight w_i is calculated to update the weights. In multivariable calculus, this rate of change is called the partial derivative. The weights are updated using the rule

$$\text{weight} = \text{weight} + r \frac{\partial L}{\partial w_i},$$

where r is the learning rate and $\frac{\partial L}{\partial w_i}$ is the partial derivative of the loss function L with respect to weight w_i .



Animation content:

A multilayer perceptron is a feed-forward network. The first step in training a model is passing the inputs and finding the predicted label or predicted value and the corresponding error or loss. This value is used in the backward propagation step to update the weights starting from the hidden layer closest to the output all the way to the input layer.

Animation captions:

1. The forward propagation step begins by feeding an instance from the training data into the neural network. Here, the instance is a five-bedroom house with 2,896 sq ft of living space.
2. Using initial values for weights and biases, the inputs pass through the neural network and an output is obtained.
3. Here, the inputs are the number of bedrooms and the total living space in square feet. The output is the predicted list price.
4. Next, the error or loss L , such as the mean square error (MSE), is calculated using the predicted values $\hat{y}_i$ and observed values y_i of the output feature.

5. The back propagation step starts with calculating the rate of change of the loss function L with respect to each weight w_i . The weights are updated starting from the output layer and moving toward the input layer.

**PARTICIPATION
ACTIVITY**

1.5.2: Multilayer perceptron.

©zyBooks 07/11/26 03:10 EDT 304020

1) A feed-forward neural network ____ a cycle between neurons.

- ☐ forms
- ☐ does not form

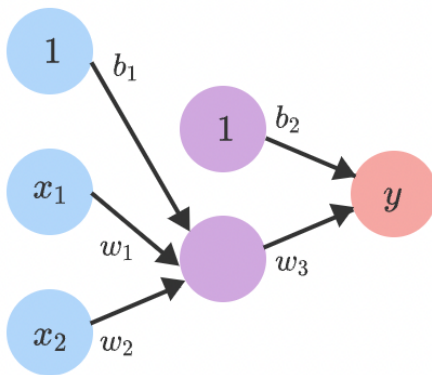
2) Multilayer perceptrons are used for ____.

- ☐ factor extraction and selection
- ☐ classification and regression
- ☐ data preprocessing

3) In the forward propagation step, the ____

- ☐ value of the output feature is predicted and the loss is computed
- ☐ partial derivative of the loss function with respect to the weights are calculated
- ☐ weights are updated starting from the output layer and moving toward the input layer.

- 4) In the backward propagation step for the neural network below, which weight is updated first?



- ☐ w_1
- ☐ w_2
- ☐ w_3

What is a loss function?

A **loss function** measures the error in a neural network. Loss functions use the difference between the expected outcome and the outcome produced by the neural network to update the weights. The type of loss function needed depends on the machine learning task. Hinge loss and log loss functions are used for classification tasks, while mean square error and mean absolute error are used for regression tasks.

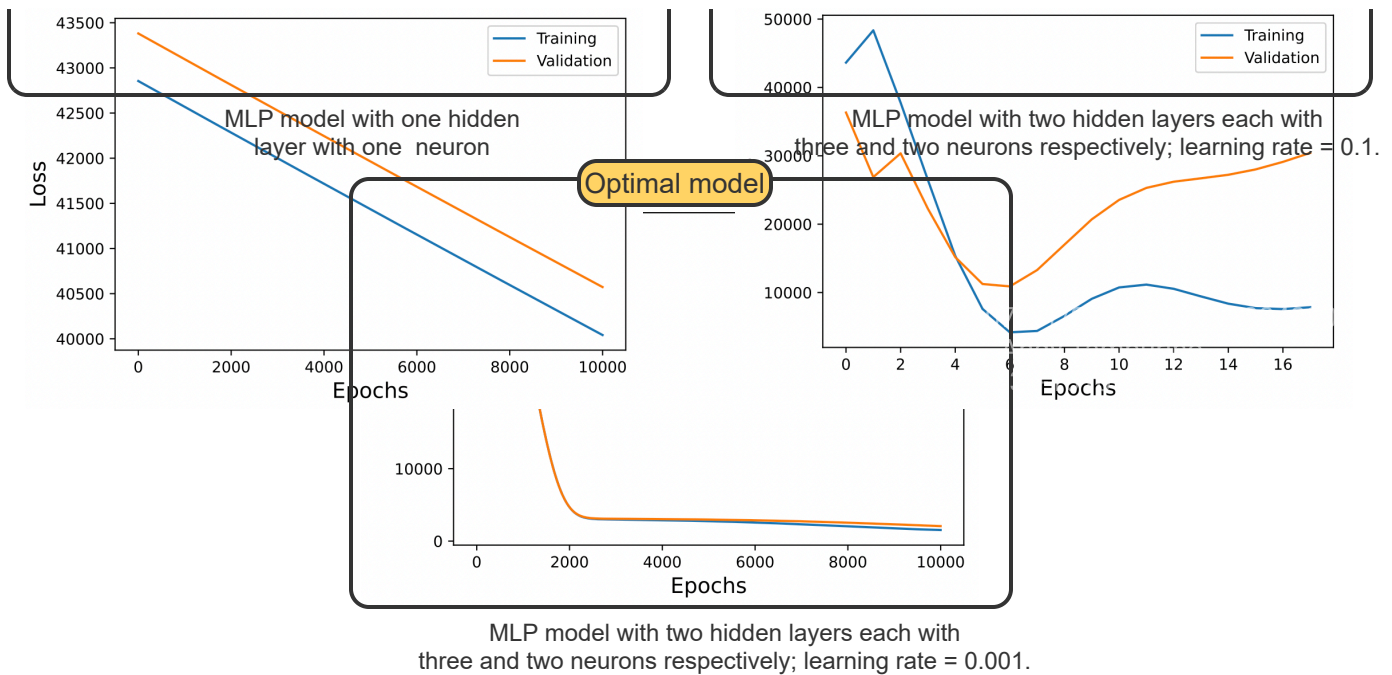
A **loss curve** measures the model error as the training of the neural network progresses. Loss curves are used to determine whether a neural network model underfits or overfits the data. Ideally, the loss curve for a neural network model should have an overall decreasing trend, although the trend can increase or decrease intermittently.

PARTICIPATION ACTIVITY

1.5.3: Using loss curves to diagnose model performance.

Underfitted model

Overfitted model



Animation content:

An underfitted MLP model with 1 hidden layer with 1 neuron has a big gap between the training and validation loss curves. Neither curve is converging to a stable loss value. An overfitted MLP model with 2 hidden layers, each with 3 and 2 neurons respectively, and a learning rate of 0.1, has training and validation loss curves that both decrease until epoch 6, but the validation curve increases without bound afterwards. An optimal MLP model with 2 hidden layers, each with 3 and 2 neurons respectively, and a learning rate of 0.001, has training and validation loss curves that both decrease to a stable loss value.

Animation captions:

1. An underfitted model can be identified by both training and validation loss curves continuing to decrease. Underfitting can be resolved by increasing the number of epochs or adding more input features.
2. An overfitted model can be identified by the validation curve attaining a minimum value and then increasing. Overfitting can be resolved by decreasing the learning rate.
3. An optimal model decreases to a stable value for loss in both training and validation. The gap between the training and validation loss curves should be minimal.



1) A loss function measures ____.

- ☐ the rate of change of the error with respect to different weights
- ☐ the rate of change of the error as the number of epochs increases
- ☐ how well a neural network performs using the difference in observed and predicted values

2) A ____ function should be used for regression tasks involving data with many outliers and influential points.

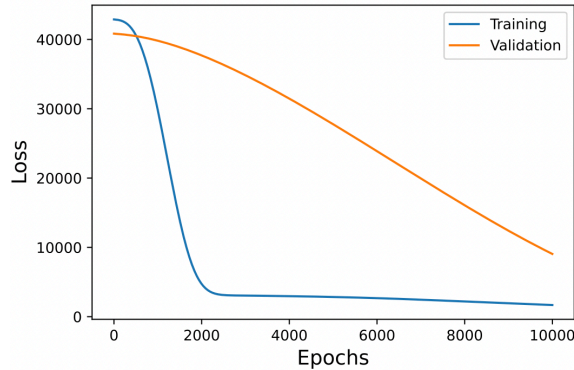
- ☐ MAE
- ☐ MSE
- ☐ hinge loss

3) An underfitted model can be improved by ____.

- ☐ increasing the number of epochs
- ☐ decreasing the number of features
- ☐ decreasing the size of the training set



- 4) Using the training and validation loss curves below, does the multilayer perceptron give optimal results?



- ☐ Yes, because both training and validation loss curves are decreasing.
- ☐ Yes, because the training loss curve decreases to a stable value.
- ☐ No, because the training and validation loss curves have a big gap.

Creating a multilayer perceptron model in Python

`MLPClassifier` and `MLPRegressor` create a multilayer perceptron model for classification and regression tasks respectively. The most common parameters are the number of epochs, `max_iter`, activation function, `activation`, and the number of neurons in each hidden layer, `hidden_layer_sizes`. The rest of the parameters and attributes can be found in the [MLPClassifier documentation](#) and [MLPRegressor documentation](#).

Table 1.5.1: Common attributes of `MLPClassifier` and `MLPRegressor` models.

Attribute	Description
<code>loss_</code>	Loss after the last epoch
<code>loss_curve_</code>	Loss at the end of each epoch
<code>coefs_</code>	Weights assigned to the input features
<code>intercepts_</code>	Weights assigned to the bias terms

Try 1.5.1: Multilayer perceptron models in Python.

[Full screen](#)

- Click the double-right arrow to restart the kernel and run all cells.
- Examine the code below.
- Change the parameter value of `hidden_layer_sizes` for the `mlpReg_train` model from `[1]` to `[3,2]`. Explore any changes in the output.



History



Start your Python (Jupyter Notebook) environment



CS-530-10101.202657-1

[Model Solution](#) ^

**PARTICIPATION
ACTIVITY**

1.5.5: Creating a multilayer perceptron model in Python.



- 1) What is the code to initialize a multilayer perceptron regressor model `m1pReg` with a maximum of 1,000 epochs and a logistic activation function?

**Check**[Show answer](#)

- 2) What is the code to display the biases given a fitted multilayer perceptron regressor model `mlpReg` after the last epoch?

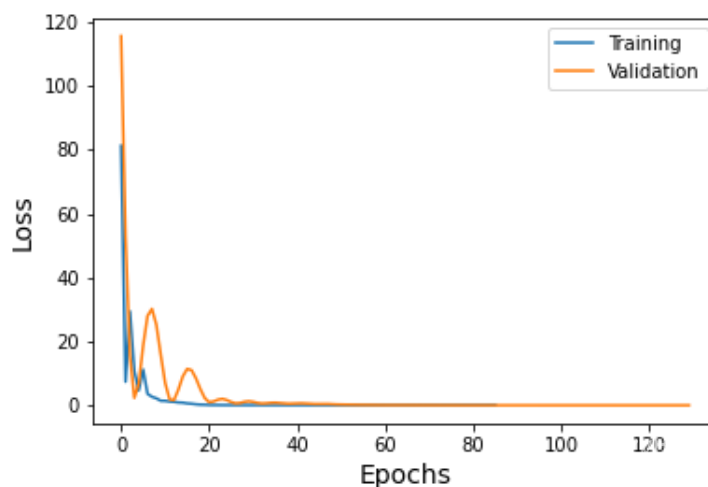
Check[Show answer](#)

- 3) What is the code to display the loss after the last epoch given a fitted multilayer perceptron regressor model `mlpReg`?

Check[Show answer](#)**CHALLENGE
ACTIVITY**

1.5.1: Multilayer perceptron.

758618.6080406.qx3zqy7

Start

Using the training and validation loss curves, does the multilayer perceptron give optimal results?



An underfitted multilayer perceptron can be improved by ____.



1

Check

Try again

Alternative text not found

**CHALLENGE
ACTIVITY**

1.5.2: Multilayer perceptron using scikit-learn.

Andy Tamburro
CS-530-10101-202657-1

758618.6080406.qx3zqy7

Start

This dataset contains information on taxi journeys during March 2019 in New York City. The data includes toll cost, total cost, and fare.

- Initialize and fit the multilayer perceptron model, `multLayerPercModel`, on:
 - the training set.
 - the validation set.

The initialization parameters for both the training and validation sets are random state set to `rng`, a maximum of 7000 epochs, and a hidden layer size of `[1, 1]`. The `np.ravel()` function is used to get the `y` data to a usable format.

The code contains all imports, loads the dataset, splits the dataset into test and train data, and prints a predicted value and the weights, biases, and loss of the fitted multilayer perceptron model.

main.py**taxisNY.csv**

```

1 # Loads necessary packages
2 import numpy as np
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.neural_network import MLPRegressor
6
7 # Seed random number generator
8 rng = np.random.RandomState(25)
9
10 # Loads the taxisNY.csv dataset
11 taxisNY = pd.read_csv('taxisNY.csv')
12
13 # Loads predictor and target variables
14 X = taxisNY[['total', 'fare']].to_numpy() # converted to numpy type array
15 y = taxisNY[['toll']]
16
17 # Splits the data into training and test sets
18 XTrain, XTest, yTrain, yTest = train_test_split(X, np.ravel(y), random_state=rng)
19
20 # Initializes and trains a multilayer perceptron regressor model on the training ar
21

```

```
22 multLayerPercModelTrain = # Your code goes here
23 multLayerPercModelValidation = # Your code goes here
24
25 # Predicts the toll cost of a taxi ride with a specific total cost and fare
26 print(multLayerPercModelTrain.predict([[36, 34]]))
27
28 # Prints the final weights, biases, and losses
29 weights = multLayerPercModelTrain.coefs_
30 biases = multLayerPercModelTrain.intercepts_
31 loss = multLayerPercModelTrain.loss_
32 print('{}\n{}\n{}'.format(weights, biases, loss))
```

1

2

3

Check

Next level

1.6 Case study: Bike share demand

Learn to:

- Explain how seasonal effects and confounding may impact a model.
- Examine multilayer perceptrons for regression and classification.
- Use performance metrics to select the best activation function.
- Implement and examine multilayer perceptrons using `scikit-learn`.

Predicting demand

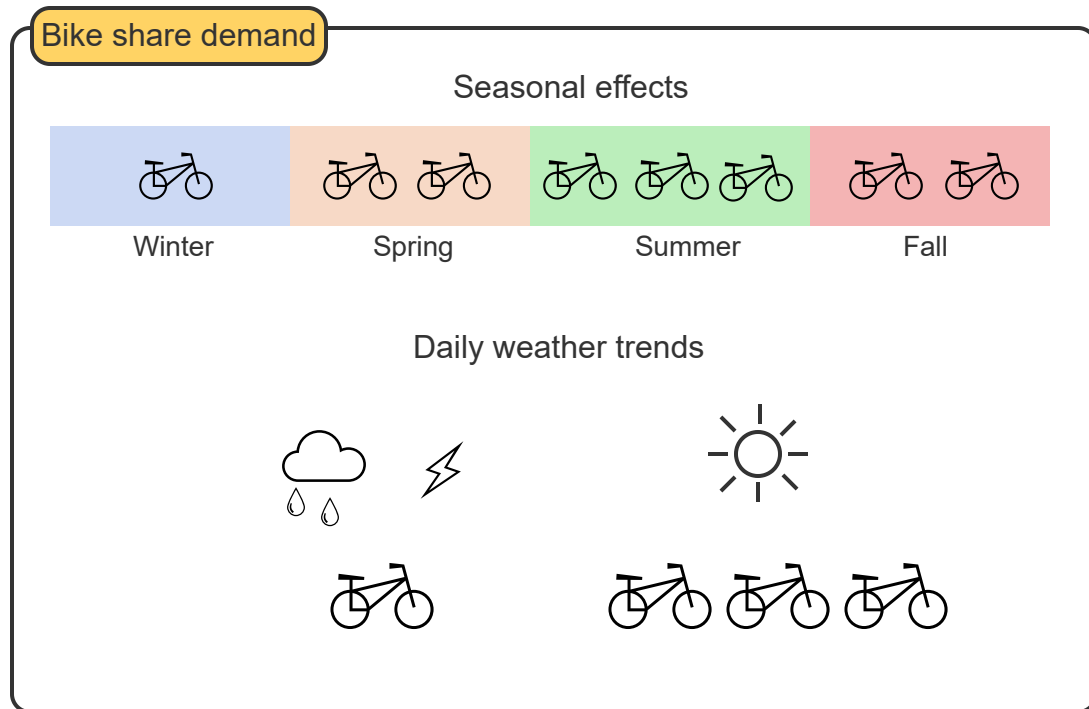
Time series data is collected over an extended time period and is affected by changes over time. Ex: Weather models use recent temperatures (time series data) to predict the daily high and low temperature. Making predictions over time comes with certain challenges. **Seasonal effects** describe patterns that

repeat over time. Ex: Consumer demand for toys increases near Christmas. Seasonal patterns usually are not linear, so simple linear regression models do not perform well with time series data.

Bike share companies allow customers to rent a bike for a short time period and return the bike to another location. Bike shares are often used by college students, commuters, and tourists. Predicting daily demand helps a bike share company ensure that bikes are available to all customers.

PARTICIPATION ACTIVITY

1.6.1: Predicting bike share demand.



Animation content:

Step 1: Bike share demand may depend on seasonal effects. Ex: Winter is snowy in some climates, which makes using a bike more difficult. A bar is shown split into four categories: Winter, Spring, Summer, and Fall. Winter contains one bike icon.

Step 2: Spring and Fall have pleasant weather and may have moderate demand. Summer has warm weather, and visiting tourists may increase demand. Spring contains two bikes, Summer contains three bikes, and Fall contains two bikes.

Step 3: Daily weather trends also affect demand. Fewer bike riders may be expected on rainy or stormy days, and more bike riders are expected on dry or sunny days. Rain and lightning icons appear next to one bike. A sunny icon appears next to three bikes.

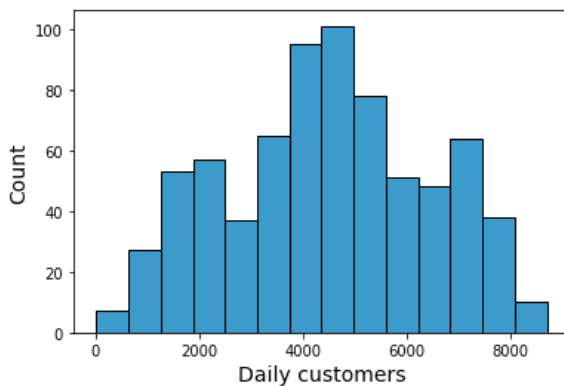
Animation captions:

1. Bike share demand may depend on seasonal effects. Ex: Winter is snowy in some climates, which makes using a bike more difficult.
2. Spring and Fall have pleasant weather and may have moderate demand. Summer has warm weather, and visiting tourists may increase demand.
3. Daily weather trends also affect demand. Fewer bike riders may be expected on rainy or stormy days, and more bike riders are expected on dry or sunny days.

PARTICIPATION ACTIVITY

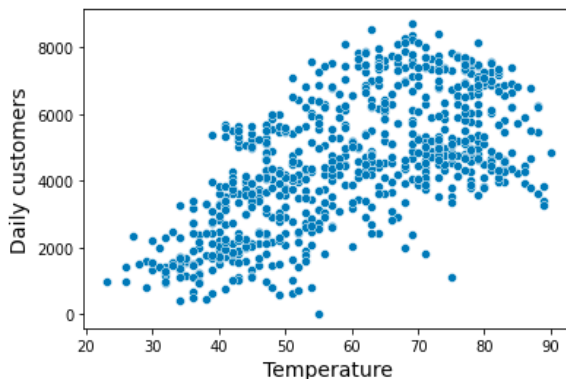
1.6.2: Exploring bike share demand.

- 1) The number of daily customers has a _____ distribution.



- ☐ skewed left
- ☐ skewed right
- ☐ symmetric

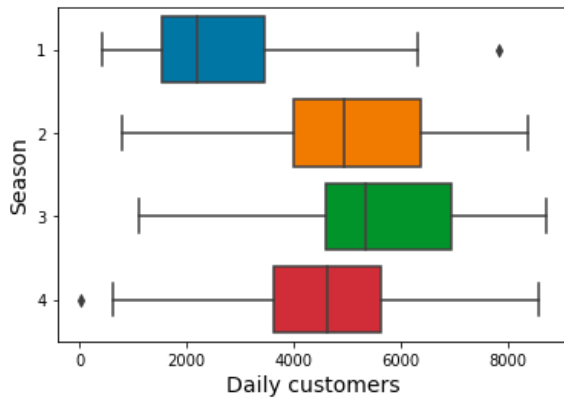
- 2) Daily customers has a _____ association with temperature.



- ☐ positive
- ☐ negative



3) Season ____ has the lowest median of daily customers.



- ☐ 1
- ☐ 2
- ☐ 3

Choosing input features for the bike share dataset

Features in a dataset are **confounded** if the features measure the same information but in a slightly different way. Ex: Day of the week (Monday-Sunday) and whether a day is a working day (Monday-Friday) or a weekend day (Saturday and Sunday) are two separate features. When confounded features exist in a dataset, only one should be included in a model.

Before applying neural network models to the bike share dataset, the features in the dataset should be explored. Features may be coded in unexpected ways. Ex: Season is recorded in the dataset as 1, 2, 3, or 4. But, season is not actually a numerical feature: 1 = Winter, 2 = Spring, 3 = Summer, and 4 = Fall. Understanding how features are coded and whether confounding is a possibility is an important part of data exploration.

Try 1.6.1: Exploring input features for predicting bike share customers.

[Full screen](#)

The Python code below imports the bike share dataset and visualizes features related to day, month, and season.

- Click the double-right arrow to restart the kernel and run all cells.
- Examine the code below.

[History](#)

Start your Python (Jupyter Notebook) environment



Model Solution [^](#)

Data source: Fanaee-T, Hadi, and Gama, Joao, "Event labeling combining ensemble detectors and background knowledge", Progress in Artificial Intelligence (2013): pp. 1-15, Springer Berlin Heidelberg, doi:10.1007/s13748-013-0040-3.

**PARTICIPATION
ACTIVITY****1.6.3: Exploring features in the bike share dataset.**

1) Which day is most likely represented by `weekday=1`?

- ☐ Sunday
- ☐ Monday
- ☐ Tuesday

2) Which feature is confounded with `weekday`?

- ☐ `workingday`
- ☐ `season`
- ☐ `month`

3) Should `month` be treated as a categorical or numerical feature?

- ☐ Categorical
- ☐ Numerical

Predicting total demand based on weather conditions

The bike share dataset contains two types of customers: casual and registered. Registered customers pay a monthly subscription fee to use the bike share services, while casual customers are one-time users. The total number of daily customers is the number of registered customers who use a bike that day plus the

number of casual customers. A model that successfully predicts total demand helps the bike share company determine how many bikes must be available for customers on a given day.

A multilayer perceptron for predicting total demand was fitted using five input features: temperature, humidity, windspeed, working day, and season. The multilayer perceptron used 70% of the original data for training and the remaining 30% for validation.

Try 1.6.2: Multilayer perceptron for predicting daily bike share customers.

[Full screen](#)

The Python code below imports the bike share dataset and fits a multilayer perceptron model to predict total daily customers.

- Click the double-right arrow to restart the kernel and run all cells.
- Examine the code below.

[History](#)

Start your Python (Jupyter Notebook) environment

[Model Solution](#) **PARTICIPATION
ACTIVITY**

1.6.4: Evaluating the multilayer perceptron.

1) Two types of multilayer perceptrons exist: one for classification and one for regression. Which multilayer perceptron was used?

- ☐ Classification
- ☐ Regression

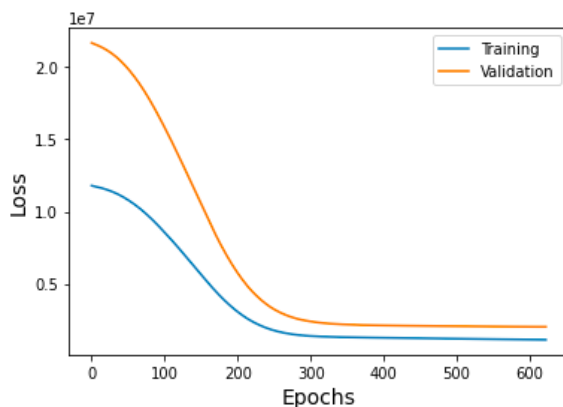
2) The multilayer perceptrons were fitted using up to 5,000 epochs, which may be time-consuming. What parameter change could decrease the number of epochs needed?

- ☐ Increase the learning rate
- ☐ Change the activation function
- ☐ Increase the validation dataset's size

3) The training dataset has a coefficient of determination, R^2 , of 0.528, which indicates ____ relationship between the actual total customers and predicted total customers.

- ☐ no
- ☐ a weak
- ☐ a strong

4) Based on the loss curves, the multilayer perceptron model is ____.



- ☐ underfitted
- ☐ overfitted
- ☐ optimally fitted

Predicting high demand from casual customers

Demand from registered customers is easier for a company to plan for: Registered customers pay a subscription fee, and the company collects revenue even if the customer does not use the service. But, casual customers are more difficult to predict. If a casual customer wants to ride a bike and none are available, the company does not collect any revenue. Understanding and predicting which days will have

higher demand from casual customers helps the bike share company make sure bikes are available when needed.

Days with more than 1,500 casual customers are classified as high demand. Three multilayer perceptron classification models are fit to predict high-demand days using different activation functions: linear, sigmoid, and ReLU.

Try 1.6.3: Multilayer perceptron for predicting high-demand days. [Full screen](#)

The Python code below imports the bike share dataset and fits a multilayer perceptron model to predict which days will have high demand from casual customers.

- Click the double-right arrow to restart the kernel and run all cells.
- Examine the code below.



 History



Start your Python (Jupyter Notebook) environment



Model Solution **PARTICIPATION
ACTIVITY**

1.6.5: Evaluating activation functions.



The bike share company fit multilayer perceptron classifiers with three activation functions to a training dataset with 511 instances and a testing dataset with 220 instances. The company wants to evaluate the models based on overall accuracy and the false negative rate. High-demand days that are not correctly predicted by the model are false negatives. 72 days in the training dataset and 37 days in the testing dataset are true high-demand days.

1) Which activation function had the highest accuracy on the testing dataset?

- ☐ Linear
- ☐ Sigmoid
- ☐ ReLU

2) Which activation function had the lowest false negative rate on the testing dataset?

- ☐ Linear
- ☐ Sigmoid
- ☐ ReLU

3) Which activation function should be chosen?

- ☐ Linear
- ☐ Sigmoid
- ☐ ReLU

1.7 LAB: Single-layer perceptron

LAB ACTIVITY

1.7.1: LAB: Single-layer perceptron

[Full screen](#)

2 / 2



The `nbaallelo_log` file contains data on 126314 NBA games from 1947 to 2015. The dataset includes the features `pts`, `elo_i`, `win_equiv`, and `game_result`. Using the csv file `nbaallelo_log.csv` and `sklearn`'s `Perceptron` function, construct a perceptron model to classify whether a team will win or lose a game based on the features `pts`, `elo_i`, `win_equiv`. Complete the program with the following tasks:

- Scale the features in `x` and `y`.
- Use the `Perceptron` function to initialize and fit a perceptron model with a learning rate of 0.05 and 20000 epochs.
- Print the weights for the input variables and bias term.
- Find the accuracy score.

Note: The program reads in a csv file's name from user.

Ex: If the program input is `nbaallelo_small.csv`, which contains 100 instances, the output is:

```
[[ 0.00479265 -0.32121666  0.37260294]]  
[0.]  
1.000
```

[Open new tab](#)[Dock](#)

▶ Run

[History](#)[Tutorial](#)

```
1  import pandas as pd  
2  import numpy as np  
3  from sklearn.linear_model import Perceptron  
4  from sklearn.model_selection import train_test_split  
5  from sklearn.preprocessing import StandardScaler  
6  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
7  
8  # Load input into a dataframe  
9  NBA = pd.read_csv(input())  
10  
11 # Hot encode the game_result variable as a numeric variable with 0 for L and 1 for W  
12 NBA.loc[NBA['game_result']=='L', 'game_result']=0  
13 NBA.loc[NBA['game_result']=='W', 'game_result']=1  
14  
15 # Store relevant columns as variables  
16 X = NBA[['pts', 'elo_i', 'win_equiv']]  
17 y = NBA[['game_result']].astype(int)  
18  
19 # Scale the input features  
20 scaler = StandardScaler()  
21 X = scaler.fit_transform(X)
```

DESKTOP

CONSOLE[Model Solution](#) ^

1.8 Artificial intelligence

Learn to:

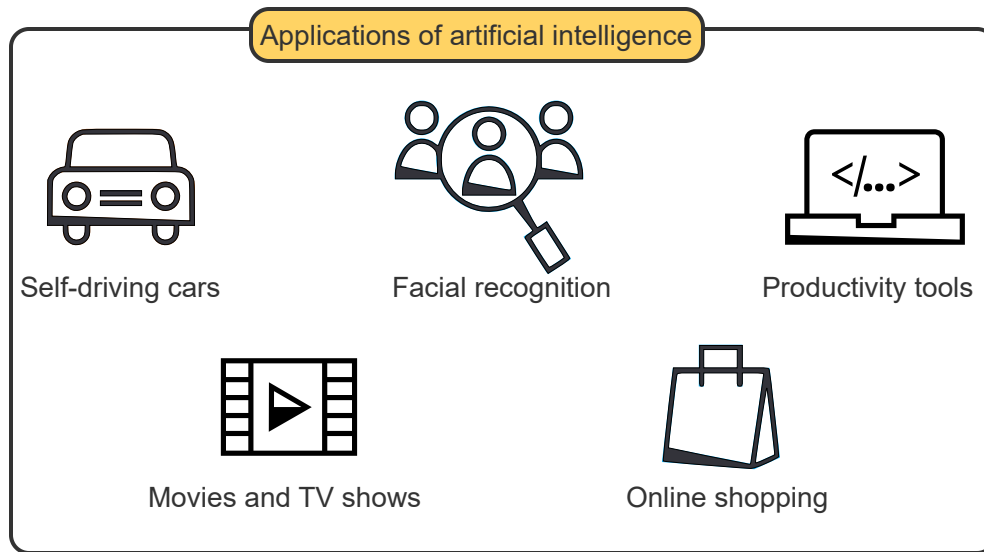
-
- Define artificial intelligence.
 - Define and identify the six domains of artificial intelligence.
 - Define deep learning.
 - Identify appropriate vs. inappropriate uses of artificial intelligence.
-

Artificial intelligence

Artificial intelligence (AI) is the development and use of algorithms and models to mimic human thought. Ex: Neural networks are one of the main models used in artificial intelligence. Artificial intelligence can be used to classify data, make predictions, or generate new outputs.

**PARTICIPATION
ACTIVITY**

1.8.1: Everyday applications of artificial intelligence.

**Animation content:**

Static image: A box labeled "Artificial intelligence" contains icons of a car, people, a computer with code, a video icon, and a shopping bag. Icons are revealed as described in the captions.

Animation captions:

1. Self-driving cars use artificial intelligence to detect obstacles and plan the car's route.
2. Facial recognition systems use artificial intelligence to recognize individuals in a crowd or from a photo.
3. Productivity tools like GitHub Copilot use artificial intelligence to help programmers write and debug code.
4. Animators and special effects artists use artificial intelligence to create backgrounds for movies and TV shows.
5. In online shopping, websites use artificial intelligence to provide customer service or recommend new products.

1) Artificial intelligence uses ____ to mimic human thought.

- ☐ algorithms and models
- ☐ decision trees
- ☐ neural networks

2) Artificial intelligence has ____ applications in the real world.

- ☐ limited
- ☐ many

3) Which types of inputs can be used in artificial intelligence?

- ☐ Text
- ☐ Video
- ☐ Images
- ☐ All of the above

4) Artificial intelligence systems ____.

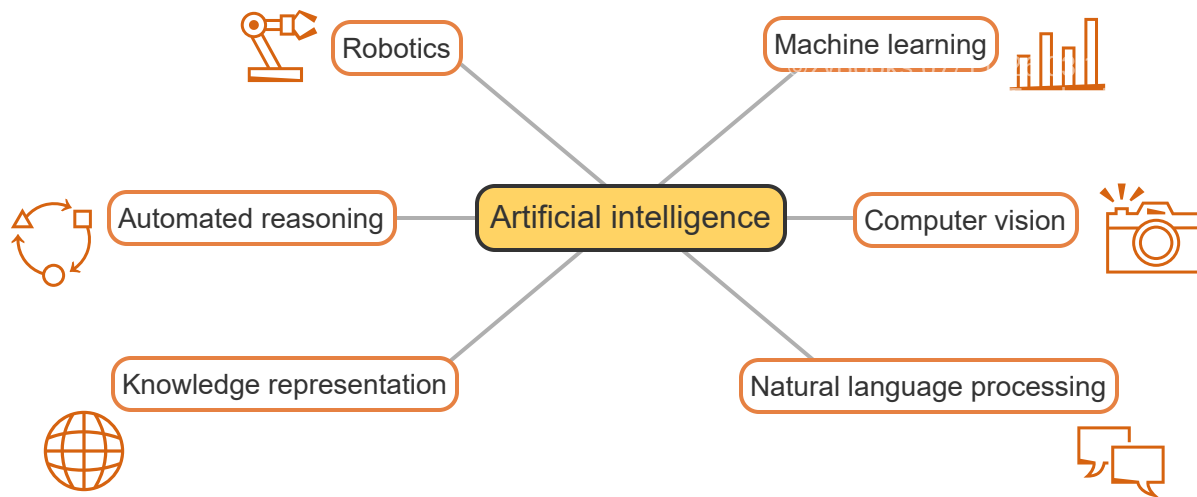
- ☐ can write a response to a question or prompt
- ☐ do not depend on training data
- ☐ have emotions or personalities

Artificial intelligence domains

Artificial intelligence contains six major domains.

- **Machine learning** uses algorithms and models to make predictions and discover patterns in data.
- **Computer vision** uses algorithms and models to extract meaning from images and video.
- **Natural language processing** uses algorithms and models to understand and interpret human language and text.
- **Knowledge representation** is a framework for representing how knowledge is stored and processed.
- **Automated reasoning** uses algorithms to reason or solve conceptual problems, such as proofs.
- **Robotics** is the design, construction, operation, and programming of robots.

Data scientists typically use methods from machine learning, computer vision, and natural language processing.



Animation content:

Static image: A hub and spoke diagram contains "Artificial intelligence" in the center with six spokes: "Machine learning", "Computer vision", "Natural language processing", "Knowledge representation", "Automated reasoning", and "Robotics". Spokes are revealed as described in the captions.

Animation captions:

1. Artificial intelligence has six major domains.
2. Machine learning uses data to classify, predict, or uncover patterns based on data. Ex: Machine learning can predict which patients are at risk for a medical condition based on health history.
3. Computer vision gains information from images and videos and generates new images. Social media platforms use computer vision to tag people or places in photos and videos.
4. Natural language processing summarizes, generates, and translates text. Browsers use natural language processing to automatically translate text on a website.
5. Knowledge representation describes the world in a way computers can understand. Video game engines use knowledge representation to generate virtual worlds and characters.
6. Automated reasoning uses algorithms for proof and deduction. Search engines use automated reasoning to generate search results.
7. In robotics, robots are designed, built, and programmed to make intelligent decisions.

PARTICIPATION
ACTIVITY

1.8.4: Artificial intelligence domains.

Match the task to the type of artificial intelligence.

How to use this tool

Automated reasoning

Robotics

Natural language processing

Machine learning

Computer vision

Knowledge representation

	Describe characteristics and similarities of an online retailer's products.
	Predict which products a customer is most likely to purchase.
	Track customer movements in a retail store.
	Design an automated system for packing orders in a warehouse.
	Plan the most efficient delivery route for an online retailer.
	Answer customer service questions with a chatbot.

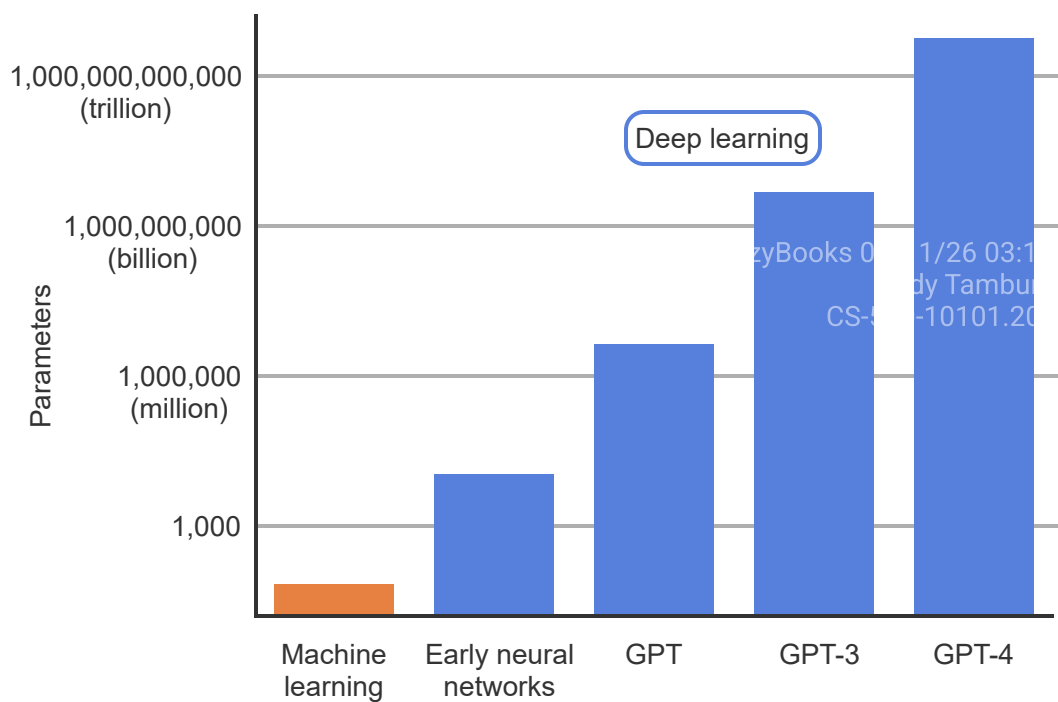
Reset

Deep learning

Deep learning describes a group of complex models with many parameters. Ex: Artificial neural networks with multiple layers are considered deep learning models. Deep learning models capture complicated relationships between input and output features, but the relationships are difficult to interpret. Until the 2000s, not enough computational resources existed to support widespread use of deep learning. But as computers have become more powerful, deep learning has integrated into modern technology.

PARTICIPATION
ACTIVITY

1.8.5: Deep learning.



Animation content:

Static image: A bar graph shows the approximate number of parameters in five models or model types. Values from the bar chart are listed in the table below.

Model/method	Parameters
Classical early machine learning	Between 0 and 1,000
Early neural networks	Between 1,000 and 1 million
GPT	Between 1 million and 1 billion
GPT-3	Between 1 billion and 1 trillion
GPT-4	Over 1 trillion

Animation captions:

- 1. Early classical machine learning models used relatively few input features, each with a unique parameter. As deep learning evolved, models grew much larger.

2. In the 1990s, neural networks were used to classify images of digits. Early neural networks had thousands of parameters.
3. In 2018, OpenAI released their first "generative pre-trained transformer," or GPT. GPT used about 117 million parameters.
4. A few years later, GPT-3 was estimated to use 175 billion parameters.
5. Released in 2023, GPT-4 was estimated to use 1.7 trillion parameters.

**PARTICIPATION
ACTIVITY**

1.8.6: Deep learning.

ZyBooks Embedded
CS-530-10101-202657-1

- 1) Deep learning models ____.
 - ☐ are interpretable
 - ☐ capture complicated relationships
 - ☐ capture simple relationships
- 2) The "deep" in deep learning refers to the model's ____.
 - ☐ inputs
 - ☐ outputs
 - ☐ layers
- 3) Deep learning is ____ domains of artificial intelligence.
 - ☐ used in all
 - ☐ used in some
 - ☐ not used in any



Using artificial intelligence

In the last few years, artificial intelligence has become more accessible. But AI's rapid growth has caused concern. A recent [Gallup poll](#) found that about 75% of Americans believe that AI will reduce the number of job opportunities. [Another poll](#) found that about 20% of Americans were worried that technology, including AI, would make their jobs obsolete. When used ethically and responsibly, AI tools can improve people's daily lives.

**PARTICIPATION
ACTIVITY**

1.8.7: Guidelines for using artificial intelligence.



Using AI tools

Check for inaccuracies

Never copy and paste

Consider potential biases

Recognize outdated results

Be transparent

Animation content:

Static image: A text box is shown with the label "Using AI tools". The following bulletpoints are listed:

Check for inaccuracies

Never copy and paste

Consider potential biases

Recognize outdated results

Be transparent

Animation captions:

1. AI tools are evolving, but some guidelines for ethical and responsible use exist.
2. AI-generated output is not always accurate and should always be reviewed for errors and inaccuracies.
3. AI-generated output should never be used verbatim. Copying and pasting AI-generated output may be plagiarism.
4. AI-generated output may contain harmful stereotypes and biases.
5. AI tools are expensive to train and create, so AI-generated output is not always up to date.
6. Users should be transparent about when AI tools are used and how.



1) Riley is taking a computer science course. Riley uses ChatGPT to answer a homework question and copies ChatGPT's output into the assignment. Is Riley's use of AI for homework appropriate?

- ☐ Yes
☐ No

2) Google recently began testing AI overviews, which return AI-generated results for search. Google lists at the top of the search result when AI overviews are used. Is Google's use of AI for search overviews appropriate?

- ☐ Yes
☐ No

3) Morgan is creating a Powerpoint presentation for work and uses Microsoft Copilot to generate a slide deck. After Copilot creates the initial deck, Morgan makes extensive edits and adds new details. Is Morgan's use of AI for generating slides appropriate?

- ☐ Yes
☐ No